Architecture And Runtime

Generated: 2026-05-24T18:25:09 Commit: `04adfbdb5a6b34d2969d67ac7e84c704c8e0915a`

AIOA Core Forensic Architecture Summary

Generated: 2026-05-24T18:25:09
Checkpoint commit: `04adfbdb5a6b34d2969d67ac7e84c704c8e0915a`
Git status at export start:

```
## main...origin/main [ahead 1]
```

Latest commit:

```
04adfbd (HEAD -> main) Checkpoint before forensic export snapshot
```

Runtime Flow

```
User input
  -> local fast routes
  -> external URL/repository boundary check
  -> local deterministic knowledge route when applicable
  -> model planning fallback
  -> structured JSON action validation
  -> human approval for non-response actions
  -> local executor
  -> operational memory/log update
  -> final response or next bounded step
```

Retrieval Architecture

AIOA currently contains two related local retrieval/control paths:

- `runtime/adaptive_routing/epistemic_kernel.py`: deterministic epistemic kernel using RHCSA search, provenance, contradiction notices, pressure score, and routing depth.
- `runtime/retrieval/linux/`: first operational deterministic Linux retrieval engine with query normalization, exact/alias/subcommand/category/family/keyword lookup, scoring, provenance attachment, and refusal behavior.

The newer retrieval engine is tested but not yet wired into the main runtime router. That is intentional and avoids premature runtime behavior changes.

Provenance Model

Source lineage is represented through:

- `runtime/knowledge/manifests/library_manifest.yaml`
- `runtime/knowledge/provenance/PROVENANCE_POLICY.md`
- `runtime/provenance_registry.json`
- `runtime/contradiction_registry.json`

Canonical Linux source:

```
runtime/knowledge/source/linux_master_library_v1.pdf
SHA256: 7eab9450dd15cc5e1607c29d9fe3b19c4cf9854bb702f113534b6ec34a34dc03
```

Legacy source remains preserved:

```
runtime/knowledge/source/RHCSA_Command_Library (1).pdf
SHA256: b8092eeabbfd80489d9e5ce8b49ba4d822aa83cc360da0a8f3c76276ac21d6b7
```

Evidence and Reasoning Separation

The architecture documents define memory as layered authority, not one generic store:

- L0 ephemeral runtime state
- L1 operational logs
- L2 reasoning traces
- L3 provenance records
- L4 immutable evidence
- L5 contradiction registry

Important boundary: runtime logs and model reasoning must not become retrieval evidence without explicit source ingestion and provenance.

Deterministic Safeguards and Feature Flags

Runtime safeguards include:

- `EPISTEMIC_KILL_SWITCH`
- `EPISTEMIC_DISABLE_MODEL`
- `EPISTEMIC_DISABLE_KNOWLEDGE_ROUTE`
- `EPISTEMIC_DISABLE_MEMORY_HATS`
- `EPISTEMIC_DISABLE_REASONING_TRACE`
- `EPISTEMIC_DISABLE_UNKNOWN_FALLBACK`

The Linux retrieval engine itself refuses low-confidence queries below the deterministic confidence threshold and does not call external APIs, embeddings, vector databases, or autonomous loops.

Execution Boundaries

`runtime/tools/executor.py` dispatches structured actions only after validation. Non-response actions require human approval in normal runtime flow. Shell execution goes through command validation/classification before dispatch.

Candidate Promotion Pipeline

Current candidate parser statistics:

Metric	Count
total parsed entries	3152
total candidate records	3152
total unique candidate commands	2570
candidate-only entries	1978
duplicates against existing canonical/index	725
internal candidate duplicates	582
malformed/unresolved entries	97

No candidate rows were promoted into canonical indexes during parsing. This is the correct safety posture.

Maturity Level

Current maturity: infrastructure prototype with strong local-first boundaries and an operational deterministic retrieval subsystem.

Implemented:

- bounded runtime loop
- approval-gated executor
- provider abstraction
- local RHCSA/Linux knowledge corpus
- canonical source manifest

- candidate parser and reports
- deterministic retrieval engine v1
- retrieval tests
- memory/provenance doctrine

Not yet implemented or intentionally deferred:

- runtime router hook for `LinuxRetrievalEngine`
- candidate promotion into canonical indexes
- reviewed alias/family expansion from candidate corpus
- full provider-independent retrieval answer renderer
- automated report packaging workflow inside repo

Known Limitations

- Retrieval paths overlap and should be unified behind one facade before router integration.
- Candidate data contains weak descriptions, path artifacts, and PDF merge artifacts.
- Runtime logs/state are present in the repository checkpoint and should receive a long-term archival/ignore policy.
- The Linux retrieval engine is intentionally not wired into the main route yet.
- The system has local-first retrieval but not a production-grade RAG/vector layer; this is by design for deterministic auditability.

Module Summaries

Generated: 2026-05-24T18:25:09 Commit: `04adfbdb5a6b34d2969d67ac7e84c704c8e0915a`

Category Inventory

Category	Files	Bytes
configuration	10	21482
docs	50	164717
governance	26	301410
knowledge	88	6742375
memory	22	105073
provenance	6	47967
reports	1	5591
repository	20	45291
retrieval	6	19916
runtime	40	124382
tests	10	47266
tooling	15	135684

Primary Modules

- `runtime/main.py`: runtime coordinator, local routes, model planning fallback, safeguards, session logging.
- `runtime/tools/executor.py`: structured action execution, approval gate, shell/filesystem/browser dispatch.
- `runtime/adaptive_routing/epistemic_kernel.py`: deterministic local epistemic control layer over RHCSA evidence.
- `runtime/retrieval/linux/`: deterministic Linux retrieval engine v1 with normalization, scoring, provenance attachment, refusal behavior.
- `runtime/knowledge/`: canonical RHCSA commands, command indexes, source PDF, extracted text, candidate index loader, reports.
- `runtime/memory/`: runtime state, evidence/reasoning trace helpers, RHCSA context injection.
- `docs/architecture/`: memory ontology, forbidden flows, access matrix.
- `MHLM_MHSR/`: governance/archive/taxonomy/case-study scaffolding for anti-hallucination analysis.

Runtime Core

Runtime coordinator, routing shell, providers, orchestrator, and adaptive routing controls.

Commit: `04adfbdb5a6b34d2969d67ac7e84c704c8e0915a`

Files in this chunk: 32

runtime/adaptive_routing/aoia_config.json

- size: 258 bytes
- sha256: `c32e3fb440e5c97903264ce143a10fb2d6e9d2036d5f1bb4f492fb577b0c9b4d`
- category: runtime

```
{
  "version": 1,
  "depths": ["shallow", "mid", "deep"],
  "pressure_thresholds": {
    "shallow_max": 33,
    "mid_max": 66
  },
  "runtime_policy": {
    "load_timing": "startup_only",
    "mutable_at_runtime": false,
    "network_required": false
  }
}
```

runtime/adaptive_routing/circadian_router.py

- size: 794 bytes
- sha256: `af4f6f7d2e03f7e6882a127f4cae0f1c42bc4f56e5f4365f54350f21c72d8c8e`
- category: runtime

```
#!/usr/bin/env python3
from __future__ import annotations

from datetime import datetime


def current_local_hour() -> int:
    """Return the current local hour as an integer from 0 to 23."""
    return datetime.now().hour


def classify_period(hour: int) -> str:
    """Classify the local hour into the first AOIA pressure window."""
    if 18 <= hour <= 23:
        return "peak_hours"
    return "off_peak_hours"


def select_routing_mode(hour: int | None = None) -> str:
    """Return the minimal DVM-inspired routing mode for the given local hour."""
    resolved_hour = current_local_hour() if hour is None else hour
    if classify_period(resolved_hour) == "peak_hours":
        return "deep_mode"
    return "surface_mode"
```

```
if __name__ == "__main__":
    print(select_routing_mode())
```

runtime/adaptive_routing/config_loader.py

- size: 2446 bytes
- sha256: 067855824055eab05a1fcd14777e759e360cfc8bf6760dbdc7baab16f78595f1
- category: runtime

```
#!/usr/bin/env python3
from __future__ import annotations

import json
from dataclasses import dataclass
from pathlib import Path
from types import MappingProxyType
from typing import Mapping


CONFIG_PATH = Path(__file__).resolve().parent / "aoia_config.json"
EXPECTED_DEPTHS = ("shallow", "mid", "deep")


@dataclass(frozen=True)
class AOIAConfig:
    """Immutable AOIA configuration loaded once by callers at startup."""

    version: int
    depths: tuple[str, str, str]
    shallow_max: int
    mid_max: int
    runtime_policy: Mapping[str, object]


def load_config(path: Path = CONFIG_PATH) -> AOIAConfig:
    """Load and validate AOIA config as a read-only object."""
    raw = json.loads(path.read_text(encoding="utf-8"))
    depths = tuple(raw.get("depths", ()))
    thresholds = raw.get("pressure_thresholds", {})
    runtime_policy = raw.get("runtime_policy", {})

    if raw.get("version") != 1:
        raise ValueError("AOIA config version must be 1")
    if depths != EXPECTED_DEPTHS:
        raise ValueError("AOIA depths must be exactly: shallow, mid, deep")
    if not isinstance(thresholds, dict):
        raise ValueError("pressure_thresholds must be an object")
    if not isinstance(runtime_policy, dict):
        raise ValueError("runtime_policy must be an object")

    shallow_max = int(thresholds.get("shallow_max"))
    mid_max = int(thresholds.get("mid_max"))
    if shallow_max < 0:
        raise ValueError("shallow_max must be >= 0")
    if mid_max <= shallow_max:
        raise ValueError("mid_max must be greater than shallow_max")
    if runtime_policy.get("load_timing") != "startup_only":
        raise ValueError("runtime_policy.load_timing must be startup_only")
```

```
    if runtime_policy.get("mutable_at_runtime") is not False:
        raise ValueError("runtime_policy.mutable_at_runtime must be false")
    if runtime_policy.get("network_required") is not False:
        raise ValueError("runtime_policy.network_required must be false")

    return AOIAConfig(
        version=1,
        depths=EXPECTED_DEPTHS,
        shallow_max=shallow_max,
        mid_max=mid_max,
        runtime_policy=MappingProxyType(dict(runtime_policy)),
    )


if __name__ == "__main__":
    config = load_config()
    print(
        f"AOIA config v{config.version}: "
        f"{config.depths[0]}<= {config.shallow_max}, "
        f"{config.depths[1]}<= {config.mid_max}, "
        f"{config.depths[2]}>{config.mid_max}"
    )
```

runtime/adaptive_routing/deterministic_router.py

- size: 474 bytes
- sha256: 0fa7bbc80d86f292cef374e4449084527367a6b7704c9060590df629b5ffaa56
- category: runtime

```
#!/usr/bin/env python3
from __future__ import annotations


def select_depth(pressure: int) -> str:
    """Return one of three deterministic routing depths for a pressure score."""
    if pressure < 0:
        raise ValueError("pressure must be >= 0")
    if pressure <= 33:
        return "shallow"
    if pressure <= 66:
        return "mid"
    return "deep"


if __name__ == "__main__":
    for sample in (0, 34, 67):
        print(f"{sample}: {select_depth(sample)}")
```

runtime/adaptive_routing/dvm_research.md

- size: 4364 bytes
- sha256: 3ec5815ad301be05fdbc2101712c243942de32d47c68f428ce9df46c5864c611
- category: runtime

```
# Adaptive Oceanic Intelligence Architecture - DVM Foundation

## Scope

This document captures the first biological reference layer for AOIA. It is not
```

an autonomous system design. It is a small research foundation for future adaptive routing based on time, energy cost, pressure, and layered operation.

Biological DVM Summary

Diel Vertical Migration (DVM) is a daily movement pattern observed in many marine and freshwater organisms, especially zooplankton and micronekton. The common pattern is:

- daylight: descend into deeper, darker water
- dusk: migrate upward
- darkness: feed closer to the surface
- dawn: descend again before visual predation risk increases

The behavior is adaptive because surface waters often contain more food, while deeper waters provide protection from predators that hunt visually.

Migration Layers

Surface layer:
- higher food availability
- higher exposure to light
- higher risk from visual predators during the day
- useful when darkness lowers predation pressure

Intermediate layer:
- transition zone between feeding and protection
- useful during dusk and dawn
- can become the preferred layer when environmental pressure is mixed

Deep layer:
- lower light
- lower visual predation risk
- often colder and metabolically cheaper
- may reduce energy use during daylight periods

Environmental Triggers

Daylight:
- increases visibility
- increases visual predation risk
- pushes many organisms toward deeper water

Darkness:
- lowers predator visibility
- enables safer feeding near the surface
- triggers upward migration in normal nocturnal DVM

Predation:
- one of the strongest selective pressures behind DVM
- organisms trade feeding opportunity against survival risk
- migration depth can increase when predator pressure is high

Energy conservation:
- deeper, colder water can reduce metabolic cost
- organisms may conserve energy by remaining deeper when feeding benefit is low
- migration itself has a cost, so movement must produce net benefit

Environmental pressure:
- oxygen, temperature, salinity, turbulence, moonlight, ice cover, and artificial light can modify migration depth and timing
- strong stratification can limit vertical movement
- low oxygen can make deep layers costly even when they are safer

Network and ecosystem conditions:
- DVM couples surface and deep food webs
- predators may track migrating prey
- carbon and nutrients are moved downward through feeding, respiration, and waste
- local ecosystem state changes whether upward movement is beneficial

Adaptive Behavior Pattern

DVM is not a simple clock. It is a recurring decision pattern shaped by:

- time of day
- light field
- risk level
- energy cost
- food availability
- physical constraints
- ecosystem feedback

The organism chooses a layer that balances opportunity and risk. That balance can shift daily, seasonally, and geographically.

AI Architecture Analogies

Deep mode:
- local/cache-first behavior
- low token usage
- reduced external calls
- conservative operation under pressure
- analogous to organisms staying deeper during high-risk daylight

Surface mode:
- high reasoning behavior
- greater external provider use
- broader context gathering
- higher token and compute cost
- analogous to organisms moving upward when opportunity outweighs risk

Transition mode, future phase:
- possible intermediate routing layer
- useful when conditions are mixed
- not implemented in this first step

Environmental pressure, future phase:
- network latency
- provider availability
- token budget
- local cache confidence
- user urgency
- system load

```
## First Implementation Report

Implemented foundation:
- created `adaptive_routing/`
- added this DVM research document
- added `routing_modes.json` with minimal routing mode definitions
- added `circadian_router.py` with local-hour based routing

Current router behavior:
- `18:00` through `23:00` maps to `deep_mode`
- all other hours map to `surface_mode`

Design constraints respected:
- no external APIs
- no autonomous agents
- no backend rewrite
- no vector database
- no distributed infrastructure
- no application-wide integration yet

Next safe expansion:
- add optional network-pressure input
- add optional token-budget input
- add tests once the routing contract is stable
- integrate into the main runtime only after explicit approval
```

runtime/adaptive_routing/environment/environment_router.py

- size: 1135 bytes
- sha256: d383188d38673edb0c0470b4f2d3f0c8f7ce560ad9f7a91ef5849a468e173764
- category: runtime

```
#!/usr/bin/env python3
from __future__ import annotations

import json
from datetime import datetime
from pathlib import Path
from typing import Any


PROFILE_PATH = Path(__file__).resolve().parent / "traffic_profiles.json"


def load_profiles(path: Path = PROFILE_PATH) -> dict[str, Any]:
    """Load static local traffic profiles."""
    return json.loads(path.read_text(encoding="utf-8"))


def current_local_hour() -> int:
    """Return the current local hour as an integer from 0 to 23."""
    return datetime.now().hour


def classify_traffic(region: str, hour: int | None = None) -> str:
    """Classify a region/hour pair as high or low traffic."""
    resolved_hour = current_local_hour() if hour is None else hour
    profiles = load_profiles()
```

```
    profile = profiles.get(region)
    if profile is None:
        raise ValueError(f"Unknown region: {region}")

    if resolved_hour in profile.get("peak_hours", []):
        return "high_traffic"
    if resolved_hour in profile.get("off_peak_hours", []):
        return "low_traffic"
    return "low_traffic"


if __name__ == "__main__":
    print(classify_traffic("Europe"))
```

runtime/adaptive_routing/environment/network_patterns.md

- size: 2674 bytes
- sha256: 9f9e357b6696fa16688e174c1604cfd5034db9cfbdc1b1f41bbbee0026929b54
- category: runtime

```
# AOIA Step 2 - Environmental Network Patterns

## Scope

This is a lightweight environmental awareness foundation for future adaptive
routing. It does not monitor live networks, call external APIs, or optimize
production traffic. It documents broad static patterns that can later inform
local-first routing decisions.

## Regional Traffic Patterns

Europe:
- work traffic commonly rises during business hours
- consumer traffic often peaks in the evening after work
- streaming, gaming, and social use commonly increase from 18:00 to 22:00
- low-traffic windows are usually late night to early morning, around 01:00 to
  05:00 local time

USA:
- business traffic follows local office hours across time zones
- residential traffic often peaks from late afternoon into evening
- streaming and gaming load commonly increases from 17:00 to 21:00 local time
- low-traffic windows are usually around 02:00 to 05:00 local time

Asia:
- usage varies strongly by country and time zone
- dense urban regions can show strong evening entertainment peaks
- mobile-first usage can keep traffic elevated later into the night
- low-traffic windows often occur around 02:00 to 05:00 local time

South America:
- work and education traffic often rises during daytime
- entertainment and messaging traffic commonly increases in the evening
- peak consumer windows often sit around 18:00 to 22:00 local time
- lower traffic commonly appears after midnight through early morning

## Infrastructure Windows
```

```
Nighttime and early morning windows are often better suited for heavier local
workloads because user demand is lower. Future AOIA routing can use these
windows for cache refresh, batch analysis, index updates, or provider-heavy
reasoning when those actions become explicitly integrated.

## AI Infrastructure Analogies

High traffic:
- conserve tokens
- prefer local cache
- delay heavy external work when possible
- avoid broad context expansion

Low traffic:
- allow deeper reasoning
- allow batch preparation
- allow larger cache maintenance tasks
- prepare knowledge for later peak windows

## Step 2 Implementation Report

Implemented:
- `adaptive_routing/environment/network_patterns.md`
- `adaptive_routing/environment/traffic_profiles.json`
- `adaptive_routing/environment/environment_router.py`

Routing logic:
- lookup the requested region in static local profiles
- if the hour is in `peak_hours`, return `high_traffic`
- if the hour is in `off_peak_hours`, return `low_traffic`
- otherwise return `low_traffic` for this first prototype

Constraints respected:
- no live monitoring
- no external APIs
- no backend integration
- no autonomous actions
- no analytics dashboard
```

runtime/adaptive_routing/environment/traffic_profiles.json

- size: 388 bytes
- sha256: 6cf87632370ac8922f038cff4f3418f7af5a18e52fe230d890fb2d4b426b8edd
- category: runtime

```
{
  "Europe": {
    "peak_hours": [18, 19, 20, 21, 22],
    "off_peak_hours": [1, 2, 3, 4, 5]
  },
  "USA": {
    "peak_hours": [17, 18, 19, 20, 21],
    "off_peak_hours": [2, 3, 4, 5]
  },
  "Asia": {
    "peak_hours": [19, 20, 21, 22, 23],
    "off_peak_hours": [2, 3, 4, 5]
  },
```

```
  "South America": {
    "peak_hours": [18, 19, 20, 21, 22],
    "off_peak_hours": [1, 2, 3, 4, 5]
  }
}
```

runtime/adaptive_routing/epistemic_kernel.py

- size: 12200 bytes
- sha256: `e5aa5f896a99eeff1b1cf7384439bc53d0f2d577f86d21a831a1bc3161efbf77`
- category: runtime

```
from __future__ import annotations

import json
import re
from dataclasses import dataclass
from pathlib import Path
from typing import Any

from adaptive_routing.deterministic_router import select_depth
from tools.epistemic_registry import (
    CONTRADICTION_REGISTRY_PATH,
    PROVENANCE_REGISTRY_PATH,
    build_contradiction_registry,
    build_provenance_registry,
    discover_knowledge_artifacts,
)
from tools.rhcsa_search import (
    TOPIC_DIRECTORIES,
    exact_command_lookup,
    grep_rhcsa,
    search_by_tag,
    search_rhcsa,
)


LINUX_OPERATIONAL_HINTS = {
    "bash",
    "chmod",
    "chown",
    "cron",
    "df",
    "dnf",
    "du",
    "firewall",
    "firewalld",
    "fstab",
    "grep",
    "journalctl",
    "linux",
    "ls",
    "lvm",
    "mkdir",
    "mount",
    "network",
    "nmcli",
    "podman",
```

```
    "ps",
    "pwd",
    "restorecon",
    "rhcsa",
    "rpm",
    "selinux",
    "service",
    "ssh",
    "sshd",
    "sudo",
    "systemctl",
    "systemd",
    "tar",
    "touch",
    "useradd",
    "vi",
    "vim",
}


@dataclass(frozen=True)
class KernelDecision:
    should_respond_locally: bool
    route: str
    depth: str
    pressure: int
    confidence: str
    response: str
    manual_review_required: bool
    manual_review_reasons: tuple[str, ...]
    evidence: tuple[dict[str, Any], ...]
    reasoning: dict[str, Any]


class AOIAEpistemicKernel:
    """Deterministic epistemic control layer for local AOIA retrieval."""

    def __init__(self, project_dir: Path) -> None:
        self.project_dir = project_dir
        self._provenance = self._load_registry(
            PROVENANCE_REGISTRY_PATH,
            lambda: build_provenance_registry(discover_knowledge_artifacts()),
        )
        self._contradictions = self._load_registry(
            CONTRADICTION_REGISTRY_PATH,
            lambda: build_contradiction_registry(discover_knowledge_artifacts()),
        )
        self._provenance_by_artifact = {
            str(record.get("artifact", "")): record
            for record in self._provenance.get("records", [])
            if isinstance(record, dict) and record.get("artifact")
        }
        self._duplicate_commands = tuple(
            item for item in self._contradictions.get("duplicate_commands", []) if isinstance(item, dict)
        )
        self._duplicate_sources = self._build_duplicate_source_index(self._duplicate_commands)
```

```
def evaluate(self, user_request: str) -> KernelDecision:
    topic_filter = self._detect_topic_filter(user_request)
    exact_results = exact_command_lookup(user_request, limit=5)
    keyword_results = search_rhcsa(user_request, limit=6, topic_filter=topic_filter)
    grep_results = grep_rhcsa(user_request, limit=4, topic_filter=topic_filter)
    tag_results = search_by_tag(user_request, limit=4, topic_filter=topic_filter)

    evidence = self._merge_results(exact_results, keyword_results, grep_results, tag_results)
    confidence = self._confidence(evidence, exact_results)
    pressure = self._pressure(user_request, evidence, exact_results, grep_results, topic_filter)
    depth = select_depth(pressure)
    contradiction_hits = self._contradiction_hits(user_request, evidence)
    manual_review_reasons = self._manual_review_reasons(confidence, contradiction_hits, evidence)
    manual_review_required = bool(manual_review_reasons)
    should_respond_locally = self._looks_linux_operational(user_request) and bool(evidence)
    route = "local_knowledge" if should_respond_locally else "model_fallback"
    response = self._format_response(
        user_request=user_request,
        route=route,
        depth=depth,
        pressure=pressure,
        confidence=confidence,
        evidence=evidence,
        contradiction_hits=contradiction_hits,
        manual_review_required=manual_review_required,
    )
    reasoning = {
        "query": user_request,
        "route": route,
        "topic_filter": topic_filter,
        "pressure": pressure,
        "depth": depth,
        "confidence": confidence,
        "evidence_count": len(evidence),
        "manual_review_required": manual_review_required,
        "manual_review_reasons": list(manual_review_reasons),
        "contradiction_count": len(contradiction_hits),
    }
    return KernelDecision(
        should_respond_locally=should_respond_locally,
        route=route,
        depth=depth,
        pressure=pressure,
        confidence=confidence,
        response=response,
        manual_review_required=manual_review_required,
        manual_review_reasons=manual_review_reasons,
        evidence=tuple(evidence),
        reasoning=reasoning,
    )

def _load_registry(self, path: Path, fallback: Any) -> dict[str, Any]:
    try:
        return json.loads(path.read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError):
        return fallback()
```

```
def _detect_topic_filter(self, user_request: str) -> str | None:
    lowered = user_request.lower()
    for topic in TOPIC_DIRECTORIES:
        normalized = topic.lower()
        if normalized in lowered:
            return topic
    return None

def _pressure(
    self,
    user_request: str,
    evidence: list[dict[str, Any]],
    exact_results: list[dict[str, Any]],
    grep_results: list[dict[str, Any]],
    topic_filter: str | None,
) -> int:
    tokens = [token for token in re.split(r"\s+", user_request.strip()) if token]
    pressure = min(len(tokens) * 4, 40)
    if self._looks_linux_operational(user_request):
        pressure += 20
    if topic_filter:
        pressure += 10
    if exact_results:
        pressure += 20
    elif grep_results:
        pressure += 10
    elif evidence:
        pressure += 5
    return min(pressure, 100)

def _merge_results(self, *result_sets: list[dict[str, Any]]) -> list[dict[str, Any]]:
    merged: dict[str, dict[str, Any]] = {}
    for result_set in result_sets:
        for item in result_set:
            path = str(item.get("file_location", "")).strip()
            if not path:
                continue
            enriched = self._enrich_evidence(item)
            current = merged.get(path)
            if current is None or int(enriched.get("score", 0)) > int(current.get("score", 0)):
                merged[path] = enriched
    return sorted(merged.values(), key=lambda item: (-int(item.get("score", 0)), item["file_location"]))

def _confidence(self, evidence: list[dict[str, Any]], exact_results: list[dict[str, Any]]) -> str:
    if exact_results:
        return "high"
    if not evidence:
        return "none"
    best_score = max(int(item.get("score", 0)) for item in evidence)
    if best_score >= 90:
        return "high"
    if best_score >= 35:
        return "medium"
    return "low"

def _enrich_evidence(self, item: dict[str, Any]) -> dict[str, Any]:
    path = str(item.get("file_location", "")).strip()
```

```
        provenance = self._provenance_by_artifact.get(path, {})
        contradictions = self._duplicate_sources.get(path, [])
        return {
            **item,
            "provenance": {
                "artifact_type": provenance.get("artifact_type", ""),
                "metadata": provenance.get("metadata", {}),
                "references": provenance.get("references", []),
                "content_hash": provenance.get("content_hash", ""),
            },
            "contradictions": contradictions,
        }

    def _contradiction_hits(self, user_request: str, evidence: list[dict[str, Any]]) -> list[dict[str, Any]]
        normalized_query = self._normalize_command(user_request)
        evidence_sources = {str(item.get("file_location", "")) for item in evidence}
        hits: list[dict[str, Any]] = []
        for duplicate in self._duplicate_commands:
            command = self._normalize_command(str(duplicate.get("command", "")))
            sources = [str(source) for source in duplicate.get("sources", []) if str(source)]
            if normalized_query and normalized_query == command:
                hits.append(duplicate)
                continue
            if evidence_sources.intersection(sources):
                hits.append(duplicate)
        deduped: dict[str, dict[str, Any]] = {}
        for hit in hits:
            key = str(hit.get("command", "")) or json.dumps(hit, sort_keys=True, ensure_ascii=False)
            deduped[key] = hit
        return list(deduped.values())

    def _manual_review_reasons(
        self,
        confidence: str,
        contradiction_hits: list[dict[str, Any]],
        evidence: list[dict[str, Any]],
    ) -> tuple[str, ...]:
        reasons: list[str] = []
        if confidence in {"low", "none"}:
            reasons.append(f"confidence_{confidence}")
        if contradiction_hits:
            reasons.append("duplicate_or_conflicting_sources_detected")
        if not evidence:
            reasons.append("no_local_evidence")
        return tuple(reasons)

    def _format_response(
        self,
        user_request: str,
        route: str,
        depth: str,
        pressure: int,
        confidence: str,
        evidence: list[dict[str, Any]],
        contradiction_hits: list[dict[str, Any]],
        manual_review_required: bool,
    ) -> str:
```

```
        lines = [
            "AOIA deterministic epistemic kernel hit.",
            f"Route: {route}",
            f"Routing depth: {depth}",
            f"Pressure score: {pressure}",
            f"Confidence: {confidence.upper()}",
            "",
        ]
        if evidence:
            lines.append("Evidence:")
            for item in evidence[:4]:
                metadata = item.get("provenance", {}).get("metadata", {})
                source_pdf = str(metadata.get("source_pdf", "")).strip()
                source_section = str(metadata.get("source_section", "")).strip()
                lines.append(f"- {item.get('topic')} [{item.get('category')}] -> {item.get('file_location')}
                if item.get("related_commands"):
                    lines.append(f"  commands: {', '.join(item.get('related_commands', [])[:5])}")
                if source_section or source_pdf:
                    provenance_bits = [part for part in (source_section, source_pdf) if part]
                    lines.append(f"  provenance: {' | '.join(provenance_bits)}")
        else:
            lines.append(f"No deterministic local evidence found for: {user_request}")

        if contradiction_hits:
            lines.append("")
            lines.append("Contradiction notices:")
            for hit in contradiction_hits[:4]:
                lines.append(f"- {hit.get('command')}: {', '.join(hit.get('sources', [])[:4])}")

        lines.append("")
        lines.append(
            "Manual review: REQUIRED"
            if manual_review_required
            else "Manual review: optional"
        )
        lines.append("Policy: contradictions are reported only; no automatic resolution is performed.")
        return "\n".join(lines).strip()

    @staticmethod
    def _build_duplicate_source_index(duplicates: tuple[dict[str, Any], ...]) -> dict[str, list[dict[str, An
        index: dict[str, list[dict[str, Any]]] = {}
        for item in duplicates:
            for source in item.get("sources", []):
                normalized = str(source).strip()
                if not normalized:
                    continue
                index.setdefault(normalized, []).append(item)
        return index

    @staticmethod
    def _normalize_command(value: str) -> str:
        return " ".join(value.strip().split()).lower()

    @staticmethod
    def _looks_linux_operational(text: str) -> bool:
        lowered = text.lower()
        if any(token in lowered for token in LINUX_OPERATIONAL_HINTS):
```

```
            return True
        return bool(re.search(r"\b[a-z0-9_-]+\s+-", lowered))
```

runtime/adaptive_routing/routing_modes.json

- size: 384 bytes
- sha256: 35365d3663495314cb6a6737e39009db9a70f737d2d3f4af7cb272632b31beb7
- category: runtime

```
{
  "deep_mode": {
    "description": "local/cache-first mode",
    "token_usage": "minimal",
    "routing_intent": "conserve energy and reduce external reasoning during peak pressure"
  },
  "surface_mode": {
    "description": "high reasoning mode",
    "token_usage": "high",
    "routing_intent": "use broader reasoning when pressure is lower or exploration is acceptable"
  }
}
```

runtime/adaptive_routing/stdout_logger.py

- size: 709 bytes
- sha256: 523443d28a68d239e0770fa1992c8c1c4e914e956600c5588d156ab420ad7392
- category: runtime

```
#!/usr/bin/env python3
from __future__ import annotations

from datetime import datetime, timezone
from uuid import uuid4


def new_correlation_id() -> str:
    """Create a short correlation id for one local AOIA decision trace."""
    return uuid4().hex[:12]


def log_event(correlation_id: str, event: str, detail: str = "") -> None:
    """Write one plain-text AOIA log line to stdout."""
    timestamp = datetime.now(timezone.utc).isoformat(timespec="seconds")
    suffix = f" detail={detail}" if detail else ""
    print(f"ts={timestamp} cid={correlation_id} event={event}{suffix}")


if __name__ == "__main__":
    cid = new_correlation_id()
    log_event(cid, "aoia_logger_ready", "stdout_only=true")
```

runtime/commands/__init__.py

- size: 174 bytes
- sha256: e20f1f4b6f07ac86cb95c254bda68f185e39008ec68f777636d2bf0e46130e3c
- category: tooling

```
from .base import CommandRegistry, CommandResult
from .local_commands import build_command_registry
```

```
__all__ = ["CommandRegistry", "CommandResult", "build_command_registry"]
```

runtime/commands/base.py

- size: 1076 bytes
- sha256: 8233ab36a5bc5f885742ce61297e1830a55536c5ea3281cdfc2d37320e4aaf0f
- category: tooling

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Callable


@dataclass
class CommandResult:
    handled: bool
    message: str = ""


CommandHandler = Callable[[str, Any], CommandResult]


class CommandRegistry:
    """Simple slash-command registry executed before any model request."""

    def __init__(self) -> None:
        self._handlers: dict[str, CommandHandler] = {}

    def register(self, name: str, handler: CommandHandler) -> None:
        self._handlers[name] = handler

    def names(self) -> list[str]:
        return sorted(self._handlers)

    def execute(self, raw_input: str, runtime: Any) -> CommandResult:
        stripped = raw_input.strip()
        if not stripped.startswith("/"):
            return CommandResult(False)

        command_text = stripped[1:]
        name, _, args = command_text.partition(" ")
        handler = self._handlers.get(name)
        if handler is None:
            return CommandResult(True, f"Unknown command: /{name}. Use /help.")
        return handler(args.strip(), runtime)
```

runtime/commands/local_commands.py

- size: 19396 bytes
- sha256: c1cfc4050d3f638161cf926f600b9f0ce45f27a6d535fff31167143b9b73c9d9
- category: tooling

```
from __future__ import annotations

import shlex
import subprocess
import sys
```

```
import zipfile
import json
from pathlib import Path

from .base import CommandRegistry, CommandResult
from tools.rhcsa_search import (
    exact_command_lookup,
    filter_by_topic,
    grep_rhcsa,
    library_status,
    load_topic,
    retrieve_examples,
    search_by_tag,
    search_rhcsa,
    search_workflows,
    suggest_related_commands,
)
from tools.validator import validate_action

SCEMDA_ZIP = Path.home() / "Desktop" / "kimi agetn..zip"


def build_command_registry() -> CommandRegistry:
    registry = CommandRegistry()
    registry.register("help", cmd_help)
    registry.register("status", cmd_status)
    registry.register("model", cmd_model)
    registry.register("vault", cmd_vault)
    registry.register("scemda", cmd_scemda)
    registry.register("tools", cmd_tools)
    registry.register("providers", cmd_providers)
    registry.register("setup", cmd_setup)
    registry.register("hat", cmd_hat)
    registry.register("scan", cmd_scan)
    registry.register("orchestrator", cmd_orchestrator)
    registry.register("worker", cmd_worker)
    registry.register("rhcsa", cmd_rhcsa)
    return registry


def cmd_help(_args: str, runtime) -> CommandResult:
    _ = runtime
    return CommandResult(
        True,
        "\n".join(
            [
                "Local commands:",
                "  /status          show local runtime state",
                "  /model           show active model and presets",
                "  /model NAME      switch model, e.g. aureon or gemini/gemini-2.5-flash",
                "  /vault           show Obsidian vault path",
                "  /providers       show cloud provider fallback status",
                "  /setup           show free API setup checklist",
                "  /hat             list/load/show/clear memory hats",
                "  /scan PATH       scan a project tree after ENTER approval",
                "  /orchestrator on|off|status",
                "  /worker status|memory|clear",
```

```
                "  /rhcsa status|savings|build|search QUERY|tag TAG|exact COMMAND|grep PATTERN|filter TOPIC
                "  /scemda ARGS     run the SCEMDA addon from Gary's zip",
                "  /tools           list registered local tools",
                "  /help            show this help",
                "  exit             quit",
            ]
        ),
    )


def cmd_status(_args: str, runtime) -> CommandResult:
    memory = runtime.memory_store.memory
    return CommandResult(
        True,
        "\n".join(
            [
                "Local runtime status:",
                f"  cwd: {memory.cwd}",
                f"  desktop: {runtime.desktop_dir}",
                f"  model: {runtime.provider_manager.describe()}",
                f"  fallback_chain: {', '.join(_fallback_chain(runtime)) or '(empty)'}",
                f"  orchestrator: {'on' if getattr(runtime, 'use_orchestrator', False) else 'off'}",
                f"  active_hat: {_active_hat_name(runtime)}",
                f"  browser_active: {memory.browser_active}",
                f"  current_url: {memory.current_browser_page or '(none)'}",
                "  local URL bootstrap: enabled",
            ]
        ),
    )


def cmd_model(args: str, runtime) -> CommandResult:
    text = args.strip()
    if not text or text.lower() in {"help", "list", "?"}:
        lines = [
            f"Current model: {runtime.provider_manager.describe()}",
            "Available presets:",
        ]
        lines.extend(f"  {line}" for line in runtime.provider_manager.available_models())
        lines.extend(
            [
                "Examples:",
                "  /model aureon",
                "  /model gemma",
                "  /model openrouter-gemma",
                "  /model openrouter/google/gemma-3-27b-it",
                "  /model gemini",
                "  /model gemini/gemini-2.5-flash",
                "  /model deepseek/deepseek-chat",
            ]
        )
        return CommandResult(True, "\n".join(lines))

    try:
        model_name = runtime.provider_manager.switch_model(text)
    except Exception as error:
        return CommandResult(True, f"Could not switch model: {error}")
```

```
    notice = runtime.provider_manager.model_notice(model_name)
    if notice:
        return CommandResult(True, f"Model switched to: {model_name}\nNote: {notice}")
    return CommandResult(True, f"Model switched to: {model_name}")


def cmd_tools(_args: str, runtime) -> CommandResult:
    tools = runtime.executor.tool_names()
    return CommandResult(True, "Registered tools:\n  " + "\n  ".join(tools))


def cmd_vault(_args: str, runtime) -> CommandResult:
    return CommandResult(True, f"Obsidian vault: {runtime.memory_store.vault_dir}")


def cmd_providers(_args: str, runtime) -> CommandResult:
    lines = ["Cloud provider fallback status:"]
    for row in runtime.provider_manager.provider_status():
        status = "ready" if row["available"] else "missing key/backend"
        enabled = "enabled" if row["enabled"] else "disabled"
        lines.append(f"  {row['full_name']} [{enabled}, {status}]")
    return CommandResult(True, "\n".join(lines))


def cmd_setup(_args: str, runtime) -> CommandResult:
    lines = [
        "Free API setup checklist:",
        "  OpenRouter Gemma: set OPENROUTER_API_KEY in ~/.config/openrouter/api.env",
        "  Gemini: set GEMINI_API_KEY in ~/.config/gemini/api.env",
        "  DeepSeek: set DEEPSEEK_API_KEY in ~/.config/deepseek/api.env",
        "  Removed from this terminal app: Ollama/local Gemma and HuggingFace.",
        "",
        "Current provider status:",
    ]
    for row in runtime.provider_manager.provider_status():
        status = "ready" if row["available"] else "missing"
        lines.append(f"  {row['full_name']}: {status}")
    return CommandResult(True, "\n".join(lines))


def cmd_hat(args: str, runtime) -> CommandResult:
    parts = args.strip().split(maxsplit=2)
    if not parts or parts[0] in {"list", "ls"}:
        active = _active_hat_name(runtime)
        lines = [f"Active memory hat: {active}", "Available memory hats:"]
        for hat in runtime.hat_store.list_hats():
            marker = "*" if hat.name == active else "-"
            lines.append(f"  {marker} {hat.name}: {hat.role}")
        return CommandResult(True, "\n".join(lines))

    command = parts[0]
    if command == "show":
        hat = runtime.hat_store.active_hat()
        if hat is None:
            return CommandResult(True, "No active memory hat.")
        return CommandResult(
            True,
```

```
            "\n".join(
                [
                    f"name: {hat.name}",
                    f"role: {hat.role}",
                    f"project_path: {hat.project_path or '(none)'}",
                    "instructions:",
                    hat.instructions,
                ]
            ),
        )

    if command == "clear":
        runtime.hat_store.clear_active()
        return CommandResult(True, "Active memory hat cleared.")

    if command == "load" and len(parts) >= 2:
        try:
            hat = runtime.hat_store.load_hat(parts[1])
        except Exception as error:
            return CommandResult(True, f"Could not load memory hat: {error}")
        return CommandResult(True, f"Loaded memory hat: {hat.name} ({hat.role})")

    if command == "save" and len(parts) >= 3:
        name = parts[1]
        instructions = parts[2]
        hat = runtime.hat_store.save_hat(
            name=name,
            role="custom",
            instructions=instructions,
            project_path=runtime.memory_store.memory.cwd,
        )
        return CommandResult(True, f"Saved memory hat: {hat.name}")

    return CommandResult(
        True,
        "Usage: /hat list | /hat load NAME | /hat show | /hat clear | /hat save NAME INSTRUCTIONS",
    )


def cmd_scan(args: str, runtime) -> CommandResult:
    path = args.strip() or runtime.memory_store.memory.cwd
    action = validate_action(
        {
            "action": "scan_project",
            "path": path,
            "reason": "Operator requested project scan.",
        }
    )
    result = runtime.executor.execute(action)
    if result.get("cancelled"):
        return CommandResult(True, "Project scan cancelled.")
    report = result.get("project_scan", {})
    lines = [
        result.get("message", "Project scan complete."),
        f"Report: {result.get('scan_report_path') or '(not written)'}",
        f"Summary: {report.get('architecture_summary', '(none)')}",
    ]
```

```
    entrypoints = report.get("entrypoints", [])
    if entrypoints:
        lines.append("Entrypoints: " + ", ".join(entrypoints[:12]))
    return CommandResult(True, "\n".join(lines))


def cmd_orchestrator(args: str, runtime) -> CommandResult:
    text = args.strip().lower()
    if text in {"on", "enable", "1", "true"}:
        runtime.enable_orchestrator(False)
        return CommandResult(
            True,
            "Orchestrator worker is disabled. Gemma/Ollama/HuggingFace were removed from this terminal app."
        )
    if text in {"off", "disable", "0", "false"}:
        runtime.enable_orchestrator(False)
        return CommandResult(True, "Gemini -> Gemma orchestrator disabled.")
    return CommandResult(
        True,
        "\n".join(
            [
                f"orchestrator: {'on' if getattr(runtime, 'use_orchestrator', False) else 'off'}",
                "workflow: disabled because Gemma/Ollama/HuggingFace are removed from this terminal build",
            ]
        ),
    )


def cmd_worker(args: str, runtime) -> CommandResult:
    text = args.strip().lower() or "status"
    if text == "clear":
        runtime.worker_memory.clear_worker_memory()
        return CommandResult(True, "Gemma worker memory cleared.")

    state = runtime.worker_memory.summarize_worker_state()
    if text == "memory":
        return CommandResult(True, json_dump(state))
    if text == "status":
        stats = state.get("token_saving_stats", {})
        lines = [
            "Gemma worker status:",
            f"  active_task: {state.get('active_task') or '(none)'}",
            f"  delegated_steps: {stats.get('delegated_steps', 0)}",
            f"  gemini_planner_calls: {stats.get('gemini_planner_calls', 0)}",
            f"  gemma_worker_calls: {stats.get('gemma_worker_calls', 0)}",
            f"  command_patterns: {len(state.get('command_patterns', []))}",
        ]
        return CommandResult(True, "\n".join(lines))
    return CommandResult(True, "Usage: /worker status | /worker memory | /worker clear")


def cmd_rhcsa(args: str, runtime) -> CommandResult:
    parts = args.strip().split(maxsplit=1)
    command = parts[0].lower() if parts else "status"
    query = parts[1] if len(parts) > 1 else ""

    if command == "status":
```

```
    status = library_status()
    lines = [
        "RHCSA local library status:",
        f"  path: {status['path']}",
        f"  exists: {status['exists']}",
        f"  files: {status['files']}",
        f"  indexed_topics: {status['indexed_topics']}",
        f"  indexed_command_names: {status['indexed_command_names']}",
        f"  indexed_command_examples: {status['indexed_command_examples']}",
        f"  indexed_workflows: {status.get('indexed_workflows', 0)}",
        f"  indexed_examples: {status.get('indexed_examples', 0)}",
        f"  size_bytes: {status['size_bytes']}",
    ]
    return CommandResult(True, "\n".join(lines))

if command == "savings":
    report_path = runtime.project_dir / "state" / "token_savings_report.json"
    if not report_path.exists():
        return CommandResult(True, "No token savings report yet.")
    try:
        payload = json.loads(report_path.read_text(encoding="utf-8"))
    except json.JSONDecodeError:
        return CommandResult(True, f"Token savings report is invalid: {report_path}")
    return CommandResult(True, json_dump(payload))

if command == "build":
    answer = input("Press ENTER to build/update the RHCSA library, or type n/cancel to reject: ").strip(
    if answer in {"n", "no", "cancel", "reject", "stop"}:
        return CommandResult(True, "RHCSA library build cancelled.")
    from tools.build_rhcsa_library import build_library

    root = build_library()
    return CommandResult(True, f"RHCSA library built at: {root}")

if command == "search":
    if not query:
        return CommandResult(True, "Usage: /rhcsa search QUERY")
    results = search_rhcsa(query, limit=8)
    return CommandResult(True, format_rhcsa_results(results))

if command == "tag":
    if not query:
        return CommandResult(True, "Usage: /rhcsa tag TAG")
    results = search_by_tag(query, limit=8)
    return CommandResult(True, format_rhcsa_results(results))

if command == "exact":
    if not query:
        return CommandResult(True, "Usage: /rhcsa exact COMMAND")
    results = exact_command_lookup(query, limit=8)
    return CommandResult(True, format_rhcsa_results(results))

if command == "grep":
    if not query:
        return CommandResult(True, "Usage: /rhcsa grep PATTERN")
    results = grep_rhcsa(query, limit=8)
    return CommandResult(True, format_rhcsa_results(results))
```

```
    if command == "filter":
        if not query or " " not in query.strip():
            return CommandResult(True, "Usage: /rhcsa filter TOPIC QUERY")
        topic_name, filtered_query = query.strip().split(maxsplit=1)
        results = filter_by_topic(topic_name, filtered_query, limit=8)
        return CommandResult(True, format_rhcsa_results(results))

    if command == "topic":
        if not query:
            return CommandResult(True, "Usage: /rhcsa topic TOPIC")
        return CommandResult(True, load_topic(query) or "RHCSA topic not found.")

    if command == "commands":
        suggestions = suggest_related_commands(query, limit=20)
        if not suggestions:
            return CommandResult(True, "No RHCSA command suggestions found.")
        lines = ["RHCSA command suggestions:"]
        for item in suggestions:
            lines.append(f"  {item['command']} [{item['topic']}]")
        return CommandResult(True, "\n".join(lines))

    if command == "workflows":
        workflows = search_workflows(query, limit=10)
        if not workflows:
            return CommandResult(True, "No RHCSA workflows found.")
        lines = ["RHCSA workflow results:"]
        for item in workflows:
            lines.append(f"  {item['topic']}: {item.get('summary', '')}")
            lines.append(f"    file: {item.get('source_file', item.get('file_location', ''))}")
        return CommandResult(True, "\n".join(lines))

    if command == "examples":
        examples = retrieve_examples(query, limit=10)
        if not examples:
            return CommandResult(True, "No RHCSA examples found.")
        lines = ["RHCSA example results:"]
        for item in examples:
            lines.append(f"  {item['topic']}: {item.get('summary', '')}")
            lines.append(f"    file: {item.get('source_file', item.get('file_location', ''))}")
        return CommandResult(True, "\n".join(lines))

    return CommandResult(
        True,
        "Usage: /rhcsa status | /rhcsa savings | /rhcsa build | /rhcsa search QUERY | /rhcsa tag TAG | /rhcs
    )


def cmd_scemda(args: str, runtime) -> CommandResult:
    addon_dir = runtime.project_dir / "addons" / "scemda"
    script_path = addon_dir / "scemda_aureon_agent_v2.py"
    extracted = ensure_scemda_addon(addon_dir)

    if not script_path.exists():
        if not SCEMDA_ZIP.exists():
            return CommandResult(
                True,
```

```
                f"SCEMDA zip not found at {SCEMDA_ZIP}. Place Gary's zip there first.",
            )
        return CommandResult(True, f"SCEMDA addon prepared at {addon_dir}, but launcher script is missing.")

    if not args.strip():
        return CommandResult(
            True,
            "\n".join(
                [
                    f"SCEMDA addon ready at: {addon_dir}",
                    f"Source zip: {SCEMDA_ZIP}",
                    "Run example:",
                    "  /scemda --start 2026-01-01 --end 2026-05-18 --out ./scemda_run --nulls 2000",
                    f"Extracted files: {len(extracted)}",
                ]
            ),
        )

    answer = input("Press ENTER to run SCEMDA, or type n/cancel to reject: ").strip().lower()
    if answer in {"n", "no", "cancel", "reject", "stop"}:
        return CommandResult(True, "SCEMDA run cancelled.")

    command = [sys.executable, str(script_path), *shlex.split(args)]
    result = subprocess.run(
        command,
        cwd=str(runtime.project_dir),
        text=True,
        capture_output=True,
        check=False,
    )
    output = "\n".join(
        [
            f"Exit code: {result.returncode}",
            result.stdout.strip() or "(no stdout)",
            result.stderr.strip() or "(no stderr)",
        ]
    ).strip()
    return CommandResult(True, output)


def _active_hat_name(runtime) -> str:
    hat = runtime.hat_store.active_hat()
    return hat.name if hat else "(none)"


def _fallback_chain(runtime) -> list[str]:
    method = getattr(runtime.provider_manager, "active_fallback_chain", None)
    if callable(method):
        return method()
    return []


def json_dump(payload) -> str:
    import json

    return json.dumps(payload, indent=2, ensure_ascii=False)
```

```
def format_rhcsa_results(results: list[dict]) -> str:
    if not results:
        return "No RHCSA results found."
    lines = ["RHCSA search results:"]
    for result in results:
        lines.append(f"  {result['topic']} [{result['category']}]")
        lines.append(f"    file: {result['file_location']}")
        if result.get("tags"):
            lines.append(f"    tags: {', '.join(result['tags'][:10])}")
        if result.get("related_commands"):
            lines.append(f"    commands: {', '.join(result['related_commands'][:8])}")
        if result.get("summary"):
            lines.append(f"    summary: {result['summary']}")
        if result.get("preview"):
            lines.append(f"    preview: {result['preview']}")
    return "\n".join(lines)


def ensure_scemda_addon(addon_dir: Path) -> list[str]:
    addon_dir.mkdir(parents=True, exist_ok=True)
    if not SCEMDA_ZIP.exists():
        return []

    extracted: list[str] = []
    wanted = {
        "README_SCEMDA_v2.md",
        "scemda_aureon_agent_v2.py",
        "scemda_comprehensive_validation.py",
        "scemda_full_validation.py",
        "scemda_open_data_fetcher.py",
    }
    with zipfile.ZipFile(SCEMDA_ZIP) as archive:
        for member in archive.namelist():
            if member not in wanted:
                continue
            target = addon_dir / Path(member).name
            with archive.open(member) as source, target.open("wb") as dest:
                dest.write(source.read())
            extracted.append(target.name)
    return extracted
```

runtime/install.sh

- size: 1974 bytes
- sha256: 5873463ae85548550ece72951c434782b0ee653f767d585d46f5ea463f530ff3
- category: runtime

```
#!/usr/bin/env bash
set -euo pipefail

PROJECT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
VENV_DIR="$PROJECT_DIR/.venv"
PYTHON_BIN="${PYTHON_BIN:-python3}"

echo "flAmeBornLLC AIOA LiGaLu installer"
echo "project_dir=$PROJECT_DIR"
```

```
if ! command -v "$PYTHON_BIN" >/dev/null 2>&1; then
  echo "ERROR: python3 not found"
  exit 1
fi

if [[ ! -d "$VENV_DIR" ]]; then
  echo "Creating virtual environment..."
  "$PYTHON_BIN" -m venv "$VENV_DIR"
else
  echo "Virtual environment already exists."
fi

echo "Upgrading pip..."
"$VENV_DIR/bin/python" -m pip install --upgrade pip

if [[ -f "$PROJECT_DIR/requirements.txt" ]]; then
  echo "Installing Python requirements..."
  "$VENV_DIR/bin/python" -m pip install -r "$PROJECT_DIR/requirements.txt"
fi

echo "Checking optional local PDF tools..."
if command -v pdftotext >/dev/null 2>&1; then
  echo "pdftotext=available"
else
  echo "WARN: pdftotext not found. Install poppler-utils to rebuild raw PDF extraction."
fi

if command -v pdfinfo >/dev/null 2>&1; then
  echo "pdfinfo=available"
else
  echo "WARN: pdfinfo not found. Install poppler-utils to inspect PDF page counts."
fi

echo "Validating bundled JSON artifacts..."
"$VENV_DIR/bin/python" -m json.tool "$PROJECT_DIR/knowledge/canonical/rhcsa_commands.json" >/dev/null
"$VENV_DIR/bin/python" -m json.tool "$PROJECT_DIR/knowledge/index/command_index.json" >/dev/null
"$VENV_DIR/bin/python" -m json.tool "$PROJECT_DIR/knowledge/context/context_pack.json" >/dev/null

echo "Compiling core Python files..."
"$VENV_DIR/bin/python" -m py_compile \
  "$PROJECT_DIR/main.py" \
  "$PROJECT_DIR/webapp.py" \
  "$PROJECT_DIR/knowledge/tools/pdf_extract.py" \
  "$PROJECT_DIR/knowledge/tools/section_parser.py" \
  "$PROJECT_DIR/knowledge/tools/canonical_builder.py" \
  "$PROJECT_DIR/knowledge/tools/index_builder.py" \
  "$PROJECT_DIR/knowledge/tools/context_pack_builder.py"

echo "Install complete."
echo "Run terminal app: ./run.sh"
echo "Run web app:      ./run_web.sh"
```

runtime/main.py

- size: 43421 bytes
- sha256: `ce852412871390833345d91644084fe98236e0bbbaf2ec23981710b81b40df4c`
- category: runtime

```python
#!/usr/bin/env python3
"""
Autonomous local runtime for shell, filesystem, and browser actions.

Architecture:
USER -> LLM -> structured JSON action -> executor -> result -> LLM -> final response
"""

from __future__ import annotations

import datetime as dt
import io
import json
import os
import re
import time
import traceback
from contextlib import redirect_stdout
from dataclasses import dataclass
from pathlib import Path
from typing import Any
from urllib.parse import parse_qs, unquote, urlparse

from commands import build_command_registry
from adaptive_routing.epistemic_kernel import AOIAEpistemicKernel
from memory.rhcsa_context import inject_linux_context
from orchestrator import GeminiGemmaOrchestrator
from orchestrator.knowledge_router import KnowledgeRouter
from providers import ProviderManager
from router import LocalRouter
from tools.executor import ExecutionEngine
from memory.gemma_worker_memory import GemmaWorkerMemory
from tools.memory_hats import MemoryHatStore
from tools.memory import MemoryStore
from tools.system_info import detect_desktop_dir
from tools.validator import extract_json_object, validate_action


PROJECT_DIR = Path(__file__).resolve().parent
PROMPT_FILE = PROJECT_DIR / "prompts" / "system_prompt.txt"
MAX_AGENT_STEPS = 8
DEBUG_RAW_RESPONSE = os.getenv("AGENT_DEBUG", "0") == "1"
MODEL_RETRY_DELAYS = (1.0, 2.0, 4.0)
EXTERNAL_URL_RE = re.compile(r"\bhttps?://\S+", re.IGNORECASE)
REPOSITORY_HOST_RE = re.compile(r"\b(?:github\.com|gitlab\.com)(?:/|\b)", re.IGNORECASE)
REPOSITORY_INTENT_RE = re.compile(
    r"\b(?:check|analy[sz]e|describe|review|inspect|sprawdz|sprawdz|przeanalizuj|opisz)\b"
    r".*\b(?:github|gitlab|repo|repository|repozytorium|projekt)\b"
    r"|\b(?:github|gitlab|repo|repository|repozytorium|projekt)\b"
    r".*\b(?:check|analy[sz]e|describe|review|inspect|sprawdz|sprawdz|przeanalizuj|opisz)\b",
    re.IGNORECASE,
)


@dataclass(frozen=True)
class EpistemicSafeguards:
    kill_switch: bool
```

```
    disable_model: bool
    disable_knowledge: bool
    disable_memory_hats: bool
    reasoning_trace_enabled: bool
    prefer_unknown: bool


def _env_flag(name: str, default: bool = False) -> bool:
    value = os.getenv(name)
    if value is None:
        return default
    return value.strip().lower() in {"1", "true", "yes", "on"}


def load_epistemic_safeguards() -> EpistemicSafeguards:
    return EpistemicSafeguards(
        kill_switch=_env_flag("EPISTEMIC_KILL_SWITCH", False),
        disable_model=_env_flag("EPISTEMIC_DISABLE_MODEL", False),
        disable_knowledge=_env_flag("EPISTEMIC_DISABLE_KNOWLEDGE_ROUTE", False),
        disable_memory_hats=_env_flag("EPISTEMIC_DISABLE_MEMORY_HATS", False),
        reasoning_trace_enabled=not _env_flag("EPISTEMIC_DISABLE_REASONING_TRACE", False),
        prefer_unknown=not _env_flag("EPISTEMIC_DISABLE_UNKNOWN_FALLBACK", False),
    )


def load_prompt_template(prompt_path: Path) -> str:
    """Read the editable runtime system prompt from disk."""
    if not prompt_path.exists():
        raise FileNotFoundError(f"Prompt file not found: {prompt_path}")
    return prompt_path.read_text(encoding="utf-8").strip()


def summarize_text(text: str, limit: int = 4000) -> str:
    """Trim long results before sending them back to the model."""
    if len(text) <= limit:
        return text
    return text[:limit] + "\n...[truncated]..."


def extract_first_url(text: str) -> str | None:
    """Extract the first HTTP(S) URL from free-form user text."""
    match = re.search(r"(?:https?|file)://\S+", text)
    if not match:
        return None
    return match.group(0).rstrip(").,!?\"'")


def normalize_external_url(raw_url: str) -> str:
    """Unwrap common redirect wrappers so the browser opens the real target."""
    parsed = urlparse(raw_url)
    host = parsed.netloc.lower()

    if host in {"l.facebook.com", "lm.facebook.com", "www.facebook.com", "facebook.com"}:
        query = parse_qs(parsed.query)
        target = query.get("u", [])
        if target:
            return unquote(target[0])
```

```
    return raw_url


def classify_external_review_request(user_input: str) -> str | None:
    """Deterministically keep external links out of local RHCSA retrieval."""
    if REPOSITORY_HOST_RE.search(user_input) or REPOSITORY_INTENT_RE.search(user_input):
        return "external_repository_review"
    if EXTERNAL_URL_RE.search(user_input):
        return "external_link_review"
    return None


def is_quota_exhausted_error(error: Exception) -> bool:
    """Detect provider quota exhaustion to avoid useless retries."""
    text = str(error)
    return "RESOURCE_EXHAUSTED" in text or "quota exceeded" in text.lower()


def is_daily_quota_error(error: Exception) -> bool:
    """Detect daily free-tier exhaustion where short retries will not help."""
    text = str(error)
    return "PerDay" in text or "free_tier_requests" in text


class AgentRuntime:
    """Main runtime loop coordinating model planning and local execution."""

    def __init__(
        self,
        provider_manager: Any,
        prompt_template: str,
        project_dir: Path,
        debug_raw: bool = False,
        max_steps: int = MAX_AGENT_STEPS,
    ) -> None:
        self.provider_manager = provider_manager
        self.prompt_template = prompt_template
        self.project_dir = project_dir
        self.debug_raw = debug_raw
        self.max_steps = max_steps
        self.safeguards = load_epistemic_safeguards()
        self.memory_store = MemoryStore(project_dir, project_dir)
        self.hat_store = MemoryHatStore(project_dir)
        self.worker_memory = GemmaWorkerMemory(project_dir)
        self.executor = ExecutionEngine(project_dir, self.memory_store)
        self.desktop_dir = detect_desktop_dir(Path.home())
        self.local_router = LocalRouter(self.desktop_dir)
        self.knowledge_router = KnowledgeRouter(project_dir)
        self.aoia_kernel = AOIAEpistemicKernel(project_dir)
        self.command_registry = build_command_registry()
        self.use_orchestrator = False
        self.orchestrator: GeminiGemmaOrchestrator | None = None
        self.session_log = (
            self.memory_store.paths.session_logs_dir
            / f"session_{self.memory_store.memory.session_id}.jsonl"
        )
```

```
def render_system_prompt(self) -> str:
    prompt = self.prompt_template
    replacements = {
        "__HOME_DIR__": str(Path.home()),
        "__DESKTOP_DIR__": str(self.desktop_dir),
        "__CURRENT_PROJECT__": str(self.project_dir),
        "__CURRENT_CWD__": self.memory_store.memory.cwd,
    }
    for key, value in replacements.items():
        prompt = prompt.replace(key, value)
    return prompt

def build_model_request(
    self,
    user_input: str,
    request_trace: list[dict[str, Any]],
) -> str:
    memory = self.memory_store.memory
    state_payload = {
        "session_id": memory.session_id,
        "cwd": memory.cwd,
        "current_task": memory.current_task,
        "previous_commands": memory.previous_commands[-10:],
        "recent_outputs": memory.recent_outputs[-6:],
        "browser_active": memory.browser_active,
        "current_browser_page": memory.current_browser_page,
        "open_tabs": memory.open_tabs[-10:],
        "screenshots": memory.screenshots[-10:],
        "desktop_dir": str(self.desktop_dir),
        "active_model": self.provider_manager.describe(),
        "fallback_chain": self._provider_fallback_chain(),
        "active_memory_hat": {} if self.safeguards.disable_memory_hats else self.hat_store.prompt_block
        "rhcsa_context": inject_linux_context(user_input),
        "obsidian_vault": str(self.memory_store.vault_dir),
        "tools": self.executor.tool_names(),
        "epistemic_safeguards": {
            "kill_switch": self.safeguards.kill_switch,
            "disable_model": self.safeguards.disable_model,
            "disable_knowledge": self.safeguards.disable_knowledge,
            "disable_memory_hats": self.safeguards.disable_memory_hats,
            "prefer_unknown": self.safeguards.prefer_unknown,
        },
        "local_fast_routes": [
            "slash commands",
            "date/status",
            "pwd/ls/curl version",
            "simple desktop folder creation",
            "URL browser bootstrap",
        ],
    }
    request_payload = {
        "user_request": user_input,
        "request_trace": request_trace,
        "instruction": (
            "Return exactly one JSON object and no markdown. "
            "Choose the next proposed action. The runtime will ask the human "
```

```
                "for ENTER approval before executing any non-respond action. "
                "Include confidence as high, medium, low, or unknown. "
                'If you do not have enough evidence, respond with "I DO NOT KNOW".'
            ),
        }

        return "\n".join(
            [
                "SYSTEM PROMPT:",
                self.render_system_prompt(),
                "",
                "RUNTIME STATE JSON:",
                json.dumps(state_payload, indent=2, ensure_ascii=False),
                "",
                "REQUEST JSON:",
                json.dumps(request_payload, indent=2, ensure_ascii=False),
            ]
        )

    def snapshot_status(self) -> dict[str, Any]:
        """Return the current runtime status for CLI and web callers."""
        memory = self.memory_store.memory
        return {
            "session_id": memory.session_id,
            "cwd": memory.cwd,
            "current_task": memory.current_task,
            "desktop_dir": str(self.desktop_dir),
            "model": self.provider_manager.describe(),
            "browser_active": memory.browser_active,
            "current_url": memory.current_browser_page,
            "open_tabs": memory.open_tabs[-10:],
            "recent_outputs": memory.recent_outputs[-10:],
            "previous_commands": memory.previous_commands[-10:],
            "session_log": str(self.session_log),
            "vault_dir": str(self.memory_store.vault_dir),
            "tools": self.executor.tool_names(),
            "active_memory_hat": self.hat_store.prompt_block(),
            "fallback_chain": self._provider_fallback_chain(),
            "provider_status": self._provider_status(),
            "orchestrator_enabled": self.use_orchestrator,
            "worker_memory": self.worker_memory.summarize_worker_state(),
            "knowledge_routing": {
                "enabled": not self.safeguards.disable_knowledge,
                "token_savings_report": str(self.knowledge_router.report_path),
                "aoia_kernel": "deterministic_local_epistemic_kernel_v0_1",
            },
            "epistemic_safeguards": {
                "kill_switch": self.safeguards.kill_switch,
                "disable_model": self.safeguards.disable_model,
                "disable_knowledge": self.safeguards.disable_knowledge,
                "disable_memory_hats": self.safeguards.disable_memory_hats,
                "reasoning_trace_enabled": self.safeguards.reasoning_trace_enabled,
                "prefer_unknown": self.safeguards.prefer_unknown,
            },
        }

    def _provider_fallback_chain(self) -> list[str]:
```

```
    method = getattr(self.provider_manager, "active_fallback_chain", None)
    if callable(method):
        return method()
    return []

def _provider_status(self) -> list[dict[str, Any]]:
    method = getattr(self.provider_manager, "provider_status", None)
    if callable(method):
        return method()
    return []

def ask_model(self, prompt: str) -> str:
    """Request one structured action from the active model provider."""
    if self.safeguards.disable_model:
        raise RuntimeError("Model planning is disabled by EPISTEMIC_DISABLE_MODEL.")
    last_error: Exception | None = None
    for attempt, delay_seconds in enumerate(MODEL_RETRY_DELAYS, start=1):
        try:
            raw_text = self.provider_manager.generate(prompt)
            if self.debug_raw:
                print("\n[DEBUG] RAW MODEL OUTPUT:")
                print(raw_text)
            return raw_text
        except Exception as error:
            last_error = error
            if is_daily_quota_error(error):
                break
            if attempt == len(MODEL_RETRY_DELAYS):
                break
            print(
                f"\n[WARN] Model request failed (attempt {attempt}/{len(MODEL_RETRY_DELAYS)}): {error}"
            )
            print(f"[WARN] Retrying in {delay_seconds:.0f}s...")
            time.sleep(delay_seconds)

    assert last_error is not None
    raise RuntimeError(f"Model request failed after retries: {last_error}")

def handle_user_request(self, user_input: str) -> None:
    """Run the bounded action loop for one user request."""
    self.memory_store.set_current_task(user_input)
    if self.safeguards.kill_switch:
        self.emit_epistemic_unknown("Epistemic kill switch is enabled.")
        return

    if self.handle_external_review_route(user_input):
        return

    if self.handle_local_route(user_input):
        return

    if self.handle_knowledge_route(user_input):
        return

    if self.use_orchestrator:
        self.handle_orchestrated_request(user_input)
        return
```

```
request_trace = self.bootstrap_local_context(user_input)

planned_actions = self.create_plan(user_input, request_trace)
if planned_actions:
    self.execute_planned_actions(planned_actions, request_trace)
    return

for step in range(1, self.max_steps + 1):
    prompt = self.build_model_request(user_input, request_trace)
    self.log_reasoning_trace(
        "model_request",
        {
            "step": step,
            "user_request": user_input,
            "prompt_preview": summarize_text(prompt, 1200),
        },
    )
    try:
        raw_output = self.ask_model(prompt)
    except Exception as error:
        self.log_error(
            {
                "step": step,
                "error": str(error),
                "traceback": traceback.format_exc(),
                "prompt_preview": summarize_text(prompt, 1200),
            }
        )
        self.handle_model_unavailable(request_trace, error)
        return

    self.log_session_event(
        "model_output",
        {
            "step": step,
            "prompt_preview": summarize_text(prompt, 1200),
            "raw_output": raw_output,
        },
    )

    try:
        action = validate_action(extract_json_object(raw_output))
    except Exception as error:
        self.log_error(
            {
                "step": step,
                "raw_output": raw_output,
                "error": str(error),
                "traceback": traceback.format_exc(),
            }
        )
        print("\n[ERROR] Invalid action JSON from model.")
        print(str(error))
        if self.safeguards.prefer_unknown:
            self.emit_epistemic_unknown("The model returned invalid structured output.")
        return
```

```
            self.print_action(action, step)

            try:
                result = self.executor.execute(action)
            except Exception as error:
                self.log_error(
                    {
                        "step": step,
                        "action": action,
                        "error": str(error),
                        "traceback": traceback.format_exc(),
                    }
                )
                print("\n[ERROR] Action execution failed.")
                print(str(error))
                return

            self.print_result(result)
            self.log_session_event(
                "step_result",
                {
                    "step": step,
                    "action": action,
                    "result": self.result_for_model(result),
                },
            )

            if action["action"] == "respond" or result.get("stop_loop"):
                return
            if result.get("cancelled"):
                return

            request_trace.append(
                {
                    "step": step,
                    "action": action,
                    "result": self.result_for_model(result),
                }
            )

        print("\nAgent> Agent stopped after reaching the maximum step limit.")

    def handle_external_review_route(self, user_input: str) -> bool:
        """Keep external URLs and repository requests out of RHCSA retrieval."""
        route = classify_external_review_request(user_input)
        if route is None:
            return False

        raw_url = extract_first_url(user_input)
        if raw_url:
            normalized_url = normalize_external_url(raw_url)
            try:
                open_result = self.executor.execute(
                    {"action": "browser_open", "url": normalized_url},
                    require_approval=False,
                )
```

```
            self.print_result(open_result)
            if open_result.get("success"):
                visible_text = self.executor.execute(
                    {"action": "browser_get_visible_text"},
                    require_approval=False,
                )
                self.print_result(visible_text)
            self.log_session_event(
                route,
                {
                    "user_request": user_input,
                    "routing_boundary": "no_rhcsa_local_knowledge",
                    "browser_handled": True,
                    "opened_url": normalized_url,
                },
            )
            return True
        except Exception as error:
            self.log_error(
                {
                    "user_request": user_input,
                    "route": route,
                    "error": str(error),
                    "traceback": traceback.format_exc(),
                }
            )
            self.log_session_event(
                route,
                {
                    "user_request": user_input,
                    "routing_boundary": "no_rhcsa_local_knowledge",
                    "browser_handled": False,
                    "opened_url": normalized_url,
                    "error": str(error),
                },
            )
            print("\nAgent> External URL detected. Browser inspection path available but browser handofi
            return True

    message = (
        "External repository inspection path detected. Browser inspection path available."
        if route == "external_repository_review"
        else "External URL detected. Browser inspection path available."
    )
    self.log_session_event(
        route,
        {
            "user_request": user_input,
            "routing_boundary": "no_rhcsa_local_knowledge",
            "browser_handled": False,
        },
    )
    print(f"\nAgent> {message}")
    return True

def enable_orchestrator(self, enabled: bool = True) -> None:
    self.use_orchestrator = enabled
```

```
    if enabled and self.orchestrator is None:
        self.orchestrator = GeminiGemmaOrchestrator(
            provider_manager=self.provider_manager,
            worker_memory=self.worker_memory,
            hat_store=self.hat_store,
            project_dir=self.project_dir,
            desktop_dir=self.desktop_dir,
            max_steps=self.max_steps,
        )

def handle_orchestrated_request(self, user_input: str) -> None:
    """Run Gemini brain -> Gemma worker -> approval -> executor flow."""
    self.enable_orchestrator(True)
    assert self.orchestrator is not None

    try:
        plan = self.orchestrator.create_plan(user_input, self.snapshot_status())
    except Exception as error:
        self.log_error(
            {
                "kind": "orchestrator_planner_error",
                **self.orchestrator.error_payload(error),
            }
        )
        print("\n[ERROR] Gemini planner failed.")
        print(str(error))
        return

    strategy = plan.get("strategy", "")
    steps = plan.get("steps", [])
    print("\n[GEMINI PLAN]")
    if strategy:
        print(strategy)
    for index, step in enumerate(steps, start=1):
        print(f"{index}. {step}")

    previous_results: list[dict[str, Any]] = []
    for index, step in enumerate(steps[: self.max_steps], start=1):
        try:
            action = self.orchestrator.action_for_step(
                user_request=user_input,
                step=step,
                runtime_status=self.snapshot_status(),
                previous_results=previous_results,
            )
        except Exception as error:
            self.log_error(
                {
                    "kind": "gemma_worker_error",
                    "step": step,
                    **self.orchestrator.error_payload(error),
                }
            )
            print("\n[ERROR] Gemma worker failed to produce a valid action.")
            print(str(error))
            print("Agent> Worker model is not available or did not return valid JSON. Use /worker status
            return
```

```
            self.print_action(action, index)
            try:
                result = self.executor.execute(action)
            except Exception as error:
                self.log_error(
                    {
                        "kind": "orchestrated_execution_error",
                        "step": step,
                        "action": action,
                        "error": str(error),
                        "traceback": traceback.format_exc(),
                    }
                )
                print("\n[ERROR] Orchestrated action execution failed.")
                print(str(error))
                return

            self.print_result(result)
            self.orchestrator.record_result(step, action, result)
            previous_results.append(
                {
                    "step": step,
                    "action": action,
                    "result": self.result_for_model(result),
                }
            )
            self.log_session_event(
                "orchestrated_step_result",
                previous_results[-1],
            )
            if action["action"] == "respond" or result.get("stop_loop") or result.get("cancelled"):
                return

    def create_plan(
        self,
        user_input: str,
        request_trace: list[dict[str, Any]],
    ) -> list[dict[str, Any]]:
        """Ask the model for a short action plan before the reactive loop."""
        prompt = self.build_plan_request(user_input, request_trace)
        self.log_reasoning_trace(
            "planner_request",
            {
                "user_request": user_input,
                "prompt_preview": summarize_text(prompt, 1200),
            },
        )
        try:
            raw_output = self.ask_model(prompt)
            payload = extract_json_object(raw_output)
        except Exception as error:
            self.log_error(
                {
                    "kind": "planner_error",
                    "error": str(error),
                    "traceback": traceback.format_exc(),
```

```
                "prompt_preview": summarize_text(prompt, 1200),
            }
        )
        return []

    raw_plan = payload.get("plan", [])
    if not isinstance(raw_plan, list):
        return []

    planned_actions: list[dict[str, Any]] = []
    for raw_action in raw_plan[: self.max_steps]:
        try:
            planned_actions.append(validate_action(raw_action))
        except Exception as error:
            self.log_error(
                {
                    "kind": "planner_action_error",
                    "raw_action": raw_action,
                    "error": str(error),
                }
            )
            return []

    if planned_actions:
        self.log_reasoning_trace(
            "planner_actions",
            {
                "user_request": user_input,
                "planned_actions": planned_actions,
            },
        )
        self.log_session_event(
            "planner_output",
            {
                "raw_output": raw_output,
                "planned_actions": planned_actions,
            },
        )
    return planned_actions

def build_plan_request(
    self,
    user_input: str,
    request_trace: list[dict[str, Any]],
) -> str:
    payload = {
        "user_request": user_input,
        "request_trace": request_trace[-4:],
        "runtime": self.snapshot_status(),
        "rhcsa_context": inject_linux_context(user_input, max_chars=3000),
        "instruction": (
            "Return exactly one JSON object with a plan array. "
            "Each plan item must be one allowed action JSON object. "
            "Keep the plan minimal and include a final respond action when the task can be completed. "
            "Do not execute anything. The runtime will require human ENTER approval before tools run."
        ),
    }
```

```
    return "\n".join(
        [
            "SYSTEM PROMPT:",
            self.render_system_prompt(),
            "",
            "PLANNER REQUEST JSON:",
            json.dumps(payload, indent=2, ensure_ascii=False),
            "",
            'EXPECTED FORMAT: {"plan":[{"action":"respond","message":"...","reason":"..."}]}',
        ]
    )

def execute_planned_actions(
    self,
    planned_actions: list[dict[str, Any]],
    request_trace: list[dict[str, Any]],
) -> None:
    print(f"\n[PLAN] {len(planned_actions)} proposed step(s).")
    for step, action in enumerate(planned_actions, start=1):
        self.print_action(action, step)
        try:
            result = self.executor.execute(action)
        except Exception as error:
            self.log_error(
                {
                    "step": step,
                    "action": action,
                    "error": str(error),
                    "traceback": traceback.format_exc(),
                }
            )
            print("\n[ERROR] Planned action execution failed.")
            print(str(error))
            return

        self.print_result(result)
        self.log_session_event(
            "planned_step_result",
            {
                "step": step,
                "action": action,
                "result": self.result_for_model(result),
            },
        )
        request_trace.append(
            {
                "step": step,
                "action": action,
                "result": self.result_for_model(result),
            }
        )
        if action["action"] == "respond" or result.get("stop_loop") or result.get("cancelled"):
            return

def run_text_request(self, user_input: str) -> dict[str, Any]:
    """Execute one text request and capture the textual transcript."""
    transcript_buffer = io.StringIO()
```

```
        with redirect_stdout(transcript_buffer):
            command_result = self.command_registry.execute(user_input, self)
            if command_result.handled:
                if command_result.message:
                    print(f"\nAgent> {command_result.message}")
            else:
                self.handle_user_request(user_input)
        transcript = transcript_buffer.getvalue().strip()
        return {
            "transcript": transcript,
            "status": self.snapshot_status(),
        }

    def handle_local_route(self, user_input: str) -> bool:
        """Execute obvious local tasks before calling the model."""
        route = self.local_router.route(user_input)
        if route is None:
            return False

        if not route.actions:
            if route.final_message:
                print(f"\nAgent> {route.final_message}")
            return True

        last_result: dict[str, Any] | None = None
        for index, raw_action in enumerate(route.actions, start=1):
            action = validate_action(raw_action)
            self.print_action(action, index)
            result = self.executor.execute(action)
            last_result = result
            self.print_result(result)
            self.log_session_event(
                "local_route_result",
                {
                    "step": index,
                    "action": action,
                    "result": self.result_for_model(result),
                },
            )

        if route.final_message:
            print(f"\nAgent> {route.final_message}")
        elif last_result and last_result.get("message"):
            print(f"\nAgent> {last_result['message']}")
        return True

    def handle_knowledge_route(self, user_input: str) -> bool:
        """Answer Linux/RHCSA operational requests from local memory first."""
        if self.safeguards.disable_knowledge:
            self.log_reasoning_trace(
                "knowledge_route_disabled",
                {"user_request": user_input},
            )
            return False
        kernel_decision = self.aoia_kernel.evaluate(user_input)
        self.log_reasoning_trace("aoia_kernel_decision", kernel_decision.reasoning)
        if kernel_decision.evidence:
```

```
            self.memory_store.append_evidence(
                "aoia_kernel_evidence",
                {
                    "query": user_input,
                    "route": kernel_decision.route,
                    "confidence": kernel_decision.confidence,
                    "manual_review_required": kernel_decision.manual_review_required,
                    "artifacts": [item.get("file_location") for item in kernel_decision.evidence],
                },
            )
        if kernel_decision.should_respond_locally:
            result = {
                "success": True,
                "message": kernel_decision.response,
                "confidence_label": kernel_decision.confidence,
                "manual_review_required": kernel_decision.manual_review_required,
                "manual_review_reasons": list(kernel_decision.manual_review_reasons),
                "stop_loop": True,
            }
            self.print_result(result)
            self.log_session_event(
                "aoia_kernel_hit",
                {
                    "confidence": kernel_decision.confidence,
                    "depth": kernel_decision.depth,
                    "pressure": kernel_decision.pressure,
                    "manual_review_required": kernel_decision.manual_review_required,
                    "evidence_count": len(kernel_decision.evidence),
                },
            )
            return True
        decision = self.knowledge_router.route(user_input, self.hat_store.prompt_block())
        if not decision.should_handle_locally:
            self.log_session_event(
                "knowledge_route_miss",
                {
                    "confidence": decision.confidence,
                    "reason": decision.reason,
                },
            )
            return False

        print(f"\nAgent> [CONFIDENCE: {decision.confidence.upper()}] {decision.response}")
        self.log_session_event(
            "knowledge_route_hit",
            {
                "confidence": decision.confidence,
                "reason": decision.reason,
                "score": decision.hit.score if decision.hit else 0,
            },
        )
        return True

    def emit_epistemic_unknown(self, reason: str) -> None:
        result = {
            "success": True,
            "message": "I DO NOT KNOW",
```

```
            "confidence_label": "UNKNOWN",
            "epistemic_note": reason,
            "stop_loop": True,
        }
        self.log_reasoning_trace(
            "unknown_response",
            {
                "reason": reason,
                "message": result["message"],
            },
        )
        self.memory_store.append_reasoning(
            "unknown_response",
            {"reason": reason, "message": result["message"]},
        )
        self.print_result(result)

    def handle_model_unavailable(
        self,
        request_trace: list[dict[str, Any]],
        error: Exception,
    ) -> None:
        """Avoid hard-crashing after partial success."""
        if is_quota_exhausted_error(error):
            print("\n[WARN] Provider quota is exhausted for the current key.")
        if request_trace:
            last_result = request_trace[-1]["result"]
            print("\n[WARN] Model became unavailable before the next planning step.")
            print(f"[WARN] {error}")
            if last_result.get("success"):
                print("Agent> Czesc operacji zostaa juz wykonana poprawnie.")
                if last_result.get("message"):
                    print(f"Agent> Ostatni zakonczony krok: {last_result['message']}")
                if last_result.get("current_url"):
                    print(f"Agent> Aktywny URL: {last_result['current_url']}")
                print("Agent> Uruchom polecenie jeszcze raz, aby dokonczyc kolejne kroki.")
                return

        print("\n[ERROR] Model is unavailable right now.")
        print(str(error))
        print("Agent> Configure a working free cloud API with /setup, or switch provider with /model.")

    def bootstrap_local_context(self, user_input: str) -> list[dict[str, Any]]:
        """Perform obvious local setup without spending model quota.

        This is intentionally narrow:
        - unwrap Facebook redirect links
        - start the browser if the request contains a URL
        - open the URL directly
        - optionally capture visible text for later analysis

        The goal is to save model requests for interpretation rather than for
        trivial browser setup.
        """
        request_trace: list[dict[str, Any]] = []
        raw_url = extract_first_url(user_input)
        if not raw_url:
```

```
            return request_trace

        normalized_url = normalize_external_url(raw_url)
        if normalized_url != raw_url:
            print(f"\n[INFO] Redirect URL unwrapped to: {normalized_url}")

        start_action = {"action": "browser_start", "reason": "Local URL bootstrap."}
        start_result = self.executor.execute(start_action)
        self.print_result(start_result)
        request_trace.append(
            {
                "step": 0,
                "action": start_action,
                "result": self.result_for_model(start_result),
            }
        )

        open_action = {
            "action": "browser_open",
            "url": normalized_url,
            "reason": "Local URL bootstrap.",
        }
        open_result = self.executor.execute(open_action)
        self.print_result(open_result)
        request_trace.append(
            {
                "step": 0,
                "action": open_action,
                "result": self.result_for_model(open_result),
            }
        )

        lowered = user_input.lower()
        if any(token in lowered for token in ("analiz", "analy", "paper", "praca", "read", "przeczy")):
            text_action = {
                "action": "browser_get_visible_text",
                "reason": "Capture visible page text before analysis.",
            }
            text_result = self.executor.execute(text_action)
            self.print_result(text_result)
            snapshot_path = self.save_page_text_snapshot(normalized_url, text_result)
            if snapshot_path is not None:
                text_result["snapshot_path"] = str(snapshot_path)
                print(f"Result: Saved text snapshot to {snapshot_path}")
            request_trace.append(
                {
                    "step": 0,
                    "action": text_action,
                    "result": self.result_for_model(text_result),
                }
            )

        return request_trace

    def save_page_text_snapshot(self, url: str, result: dict[str, Any]) -> Path | None:
        """Persist locally captured page text so quota failures do not lose context."""
        text = result.get("text", "").strip()
```

```
    if not text:
        return None

    parsed = urlparse(url)
    slug = parsed.netloc.replace(".", "_") or "page"
    timestamp = dt.datetime.now().strftime("%Y%m%d_%H%M%S")
    snapshot_path = self.memory_store.paths.memory_dir / f"{slug}_{timestamp}.txt"
    snapshot_path.write_text(text, encoding="utf-8")
    return snapshot_path

def result_for_model(self, result: dict[str, Any]) -> dict[str, Any]:
    payload = dict(result)
    if "stdout" in payload:
        payload["stdout"] = summarize_text(str(payload["stdout"]), 2500)
    if "stderr" in payload:
        payload["stderr"] = summarize_text(str(payload["stderr"]), 2500)
    if "content" in payload:
        payload["content"] = summarize_text(str(payload["content"]), 2500)
    if "text" in payload:
        payload["text"] = summarize_text(str(payload["text"]), 2500)
    if "html" in payload:
        payload["html"] = summarize_text(str(payload["html"]), 2500)
    if "matches" in payload:
        payload["matches"] = payload["matches"][:20]
    return payload

def log_session_event(self, kind: str, payload: dict[str, Any]) -> None:
    record = {
        "timestamp": dt.datetime.now().isoformat(),
        "kind": kind,
        "payload": payload,
    }
    with self.session_log.open("a", encoding="utf-8") as handle:
        handle.write(json.dumps(record, ensure_ascii=False) + "\n")

def log_reasoning_trace(self, kind: str, payload: dict[str, Any]) -> None:
    if not self.safeguards.reasoning_trace_enabled:
        return
    self.memory_store.append_reasoning(kind, payload)

def log_error(self, payload: dict[str, Any]) -> None:
    error_file = (
        self.memory_store.paths.error_logs_dir
        / f"error_{dt.datetime.now().strftime('%Y%m%d_%H%M%S_%f')}.json"
    )
    error_file.write_text(json.dumps(payload, indent=2, ensure_ascii=False), encoding="utf-8")

def print_action(self, action: dict[str, Any], step: int) -> None:
    print(f"\n[STEP {step}] action={action['action']}")
    if action.get("reason"):
        print(f"Reason: {action['reason']}")
    for field in ("command", "path", "url", "selector", "key"):
        if field in action and action[field]:
            print(f"{field}: {action[field]}")

def print_result(self, result: dict[str, Any]) -> None:
    if result.get("confidence_label"):
```

```
            print(f"[CONFIDENCE: {str(result['confidence_label']).upper()}]")
        if result.get("manual_review_required"):
            print("[MANUAL REVIEW: REQUIRED]")
            reasons = result.get("manual_review_reasons") or []
            if reasons:
                print(f"Review reasons: {', '.join(str(reason) for reason in reasons)}")
        if result.get("message"):
            prefix = "Agent>" if result.get("stop_loop") else "Result:"
            print(f"{prefix} {result['message']}")
        if result.get("epistemic_note"):
            print(f"Epistemic note: {result['epistemic_note']}")

        if "stdout" in result:
            print("\n--- STDOUT ---")
            print(result["stdout"] if result["stdout"].strip() else "(empty)")

        if "stderr" in result:
            print("\n--- STDERR ---")
            print(result["stderr"] if result["stderr"].strip() else "(empty)")

        if "content" in result:
            print("\n--- FILE CONTENT ---")
            print(result["content"])

        if "text" in result:
            print("\n--- PAGE TEXT ---")
            print(result["text"])

        if "current_url" in result and result["current_url"]:
            print(f"\nCurrent URL: {result['current_url']}")

        if "screenshot_path" in result:
            print(f"Screenshot: {result['screenshot_path']}")

        if "exit_code" in result:
            print(f"\nExit code: {result['exit_code']}")


def print_banner(runtime: AgentRuntime) -> None:
    print("############################################################")
    print("###  flAmeBornLLC  |  LLM Academy                      ###")
    print("###  LOCAL AI TERMINAL + BROWSER AGENT                 ###")
    print("############################################################")
    print(f"[INFO] Desktop directory detected: {runtime.desktop_dir}")
    print(f"[INFO] Current working directory: {runtime.memory_store.memory.cwd}")
    print(f"[INFO] Active model: {runtime.provider_manager.describe()}")
    print(f"[INFO] Session log: {runtime.session_log}")
    print(f"[INFO] Obsidian vault: {runtime.memory_store.vault_dir}")


def main() -> None:
    provider_manager = ProviderManager(PROJECT_DIR)
    prompt_template = load_prompt_template(PROMPT_FILE)
    runtime = AgentRuntime(
        provider_manager=provider_manager,
        prompt_template=prompt_template,
        project_dir=PROJECT_DIR,
```

```
        debug_raw=DEBUG_RAW_RESPONSE,
    )

    print_banner(runtime)

    while True:
        try:
            user_input = input("\nYou> ").strip()

            if not user_input:
                continue

            if user_input.lower() in {"exit", "quit", "q"}:
                print("Exiting agent...")
                break

            command_result = runtime.command_registry.execute(user_input, runtime)
            if command_result.handled:
                if command_result.message:
                    print(f"\nAgent> {command_result.message}")
                continue

            runtime.handle_user_request(user_input)
        except KeyboardInterrupt:
            print("\nInterrupted by user.")
            break
        except Exception as error:
            runtime.log_error(
                {
                    "error": str(error),
                    "traceback": traceback.format_exc(),
                }
            )
            print(f"\n[FATAL ERROR] {error}")
            break


if __name__ == "__main__":
    main()
```

runtime/orchestrator/__init__.py

- size: 89 bytes
- sha256: 51584e8c9df8305d3bf3c0803c2cacbca2d88bc051b7701ea8d1f3be176485a8
- category: runtime

```
from .gemini_gemma import GeminiGemmaOrchestrator

__all__ = ["GeminiGemmaOrchestrator"]
```

runtime/orchestrator/gemini_gemma.py

- size: 7465 bytes
- sha256: e493ea2af9138be8a3f0aaaefb11ad81cbc18ea26f3d122ccbf52dad51056993
- category: runtime

```
from __future__ import annotations

import json
```

```python
import traceback
from pathlib import Path
from typing import Any

from memory.rhcsa_context import (
    inject_linux_context,
    retrieve_command_patterns,
    retrieve_operational_examples,
)
from tools.memory_hats import MemoryHatStore
from tools.validator import extract_json_object, validate_action


class GeminiGemmaOrchestrator:
    """Delegates strategic planning to Gemini and action generation to Gemma."""

    def __init__(
        self,
        provider_manager: Any,
        worker_memory: Any,
        hat_store: MemoryHatStore,
        project_dir: Path,
        desktop_dir: Path,
        max_steps: int = 8,
    ) -> None:
        self.provider_manager = provider_manager
        self.worker_memory = worker_memory
        self.hat_store = hat_store
        self.project_dir = project_dir
        self.desktop_dir = desktop_dir
        self.max_steps = max_steps
        self.gemma_provider = None

    def create_plan(self, user_request: str, runtime_status: dict[str, Any]) -> dict[str, Any]:
        self.worker_memory.record_gemini_call()
        prompt = self._build_gemini_planner_prompt(user_request, runtime_status)
        raw = self.provider_manager.generate_with_fallback(prompt)
        payload = extract_json_object(raw)
        raw_steps = payload.get("steps", payload.get("plan", []))
        if not isinstance(raw_steps, list):
            raw_steps = []
        steps = [str(step).strip() for step in raw_steps if str(step).strip()][: self.max_steps]
        if not steps and payload.get("message"):
            steps = [f"respond to user: {payload['message']}"]
        return {
            "strategy": str(payload.get("strategy", "")).strip(),
            "steps": steps,
            "raw": raw,
        }

    def action_for_step(
        self,
        user_request: str,
        step: str,
        runtime_status: dict[str, Any],
        previous_results: list[dict[str, Any]],
    ) -> dict[str, Any]:
```

```
    if self.gemma_provider is None:
        raise RuntimeError("Gemma/Ollama/HuggingFace worker is disabled in this terminal build.")
    self.worker_memory.record_gemma_call()
    prompt = self._build_gemma_worker_prompt(user_request, step, runtime_status, previous_results)
    raw = self.gemma_provider.generate(prompt)
    action = validate_action(extract_json_object(raw))
    self.worker_memory.remember_step(
        delegated_step=step,
        action=action,
        result=None,
        gemini_instruction=runtime_status.get("current_task", user_request),
    )
    return action

def fallback_action_for_step(
    self,
    user_request: str,
    step: str,
    previous_results: list[dict[str, Any]],
) -> dict[str, Any]:
    """Create a conservative action if Gemma is unavailable."""
    lower = step.lower()
    if "scan" in lower or "inspect" in lower or "repository" in lower or "project" in lower:
        return {
            "action": "scan_project",
            "path": str(self.project_dir),
            "reason": f"Fallback action for delegated step: {step}",
        }
    if "folder" in lower and "desktop" in lower:
        return {
            "action": "create_folder",
            "path": str(self.desktop_dir / "AI_PROJECT"),
            "reason": f"Fallback action for delegated step: {step}",
        }
    repeated_step = bool(previous_results and step == previous_results[-1].get("step", ""))
    if "respond" in lower or repeated_step:
        return {
            "action": "respond",
            "message": "Delegated plan step completed as far as the available tools allowed.",
            "reason": f"Fallback response for delegated step: {step}",
        }
    return {
        "action": "respond",
        "message": f"Gemma worker is unavailable. Planned step needs manual follow-up: {step}",
        "reason": "Fallback response because no worker model produced a valid JSON action.",
    }

def record_result(self, step: str, action: dict[str, Any], result: dict[str, Any]) -> None:
    self.worker_memory.remember_step(
        delegated_step=step,
        action=action,
        result=result,
        gemini_instruction=self.worker_memory.last_gemini_instruction,
    )

def error_payload(self, error: Exception) -> dict[str, str]:
    return {
```

```
            "error": str(error),
            "traceback": traceback.format_exc(),
        }

    def _build_gemini_planner_prompt(self, user_request: str, runtime_status: dict[str, Any]) -> str:
        hat = self.hat_store.prompt_block()
        payload = {
            "role": "Gemini Brain / Teacher / Planner",
            "task": user_request,
            "runtime_status": {
                "cwd": runtime_status.get("cwd"),
                "desktop_dir": runtime_status.get("desktop_dir"),
                "active_memory_hat": hat,
                "rhcsa_context": inject_linux_context(user_request, max_chars=3000),
                "recent_outputs": runtime_status.get("recent_outputs", [])[-4:],
            },
            "instruction": (
                "Create a short strategic plan. Do not generate executable JSON actions. "
                "Return JSON only: {\"strategy\":\"...\",\"steps\":[\"step 1\", \"step 2\"]}. "
                "Gemma worker will convert one step at a time into approved executable actions."
            ),
        }
        return json.dumps(payload, indent=2, ensure_ascii=False)

    def _build_gemma_worker_prompt(
        self,
        user_request: str,
        step: str,
        runtime_status: dict[str, Any],
        previous_results: list[dict[str, Any]],
    ) -> str:
        payload = {
            "role": "Gemma Worker / Executor JSON Action Generator",
            "user_request": user_request,
            "delegated_step": step,
            "worker_memory": self.worker_memory.summarize_worker_state(),
            "operational_command_patterns": retrieve_command_patterns(
                f"{user_request} {step}",
                limit=8,
            ),
            "operational_examples": retrieve_operational_examples(
                f"{user_request} {step}",
                limit=3,
            ),
            "runtime": {
                "cwd": runtime_status.get("cwd"),
                "desktop_dir": runtime_status.get("desktop_dir"),
                "project_dir": str(self.project_dir),
                "tools": runtime_status.get("tools", []),
            },
            "previous_results": previous_results[-3:],
            "instruction": (
                "Convert the delegated step into exactly one valid action JSON object. "
                "Use only the available tool names. Do not explain. Do not use markdown. "
                "The runtime will request human ENTER approval before execution."
            ),
            "examples": [
```

```
                {"action": "scan_project", "path": str(self.project_dir), "reason": "Inspect repository stru
                {"action": "create_folder", "path": str(self.desktop_dir / "test_ai"), "reason": "Create rec
                {"action": "respond", "message": "Task complete.", "reason": "No more actions are required."
            ],
        }
        return json.dumps(payload, indent=2, ensure_ascii=False)
```

runtime/orchestrator/knowledge_router.py

- size: 5298 bytes
- sha256: c6ab43eadcd61171a55f01634e492bb648e956d439d4f1a06fa38301634bb4d6
- category: runtime

```
from __future__ import annotations

import datetime as dt
import json
from dataclasses import dataclass
from pathlib import Path
from typing import Any

from knowledge.rhcsa_engine import KnowledgeHit, RHCSAKnowledgeEngine


LINUX_OPERATIONAL_HINTS = {
    "bash",
    "boot",
    "chmod",
    "chown",
    "cron",
    "dnf",
    "firewall",
    "firewalld",
    "fstab",
    "journalctl",
    "linux",
    "lvm",
    "mount",
    "network",
    "nginx",
    "nmcli",
    "podman",
    "rhel",
    "rhcsa",
    "root password",
    "selinux",
    "service",
    "ssh",
    "sshd",
    "systemctl",
    "systemd",
    "useradd",
}


@dataclass
class KnowledgeDecision:
    should_handle_locally: bool
```

```
    confidence: str
    reason: str
    response: str
    hit: KnowledgeHit | None


class KnowledgeRouter:
    """Decides whether local RHCSA memory can answer before API reasoning."""

    def __init__(self, project_dir: Path, engine: RHCSAKnowledgeEngine | None = None) -> None:
        self.project_dir = project_dir
        self.engine = engine or RHCSAKnowledgeEngine(project_dir)
        self.report_path = project_dir / "state" / "token_savings_report.json"
        self.report_path.parent.mkdir(parents=True, exist_ok=True)
        self._ensure_report()

    def route(self, user_request: str, active_hat: dict[str, Any] | None = None) -> KnowledgeDecision:
        if not self._looks_linux_operational(user_request, active_hat):
            return KnowledgeDecision(False, "none", "not_linux_operational", "", None)

        hit = self.engine.retrieve_operational_memory(user_request)
        prefer_local = self._hat_prefers_local(active_hat)
        threshold = {"linux": "low", "coding": "medium", "research": "high"}.get(
            str((active_hat or {}).get("name", "")).lower(),
            "medium",
        )
        if prefer_local:
            threshold = "low"

        if self._meets_threshold(hit.confidence, threshold):
            response = self.engine.format_local_answer(hit)
            self.record_local_hit(hit, avoided_reason="local_rhcsa_memory")
            return KnowledgeDecision(True, hit.confidence, "local_rhcsa_memory", response, hit)

        self.record_miss(user_request, hit.confidence)
        return KnowledgeDecision(False, hit.confidence, "low_confidence_local_memory", "", hit)

    def record_local_hit(self, hit: KnowledgeHit, avoided_reason: str) -> None:
        report = self._read_report()
        report["api_calls_avoided"] = int(report.get("api_calls_avoided", 0)) + 1
        report["local_retrieval_hits"] = int(report.get("local_retrieval_hits", 0)) + 1
        report["command_reuse_frequency"] = int(report.get("command_reuse_frequency", 0)) + len(hit.commands
        report["workflow_reuse"] = int(report.get("workflow_reuse", 0)) + len(hit.workflows)
        report["last_hit"] = {
            "timestamp": dt.datetime.now().isoformat(),
            "query": hit.query,
            "confidence": hit.confidence,
            "score": hit.score,
            "reason": avoided_reason,
        }
        self._write_report(report)

    def record_miss(self, query: str, confidence: str) -> None:
        report = self._read_report()
        report["local_retrieval_misses"] = int(report.get("local_retrieval_misses", 0)) + 1
        report["last_miss"] = {
            "timestamp": dt.datetime.now().isoformat(),
```

```
            "query": query,
            "confidence": confidence,
        }
        self._write_report(report)

    def _looks_linux_operational(self, text: str, active_hat: dict[str, Any] | None) -> bool:
        lowered = text.lower()
        if self._hat_prefers_local(active_hat):
            return any(token in lowered for token in LINUX_OPERATIONAL_HINTS)
        return any(token in lowered for token in LINUX_OPERATIONAL_HINTS)

    @staticmethod
    def _hat_prefers_local(active_hat: dict[str, Any] | None) -> bool:
        if not active_hat:
            return False
        name = str(active_hat.get("name", "")).lower()
        role = str(active_hat.get("role", "")).lower()
        instructions = str(active_hat.get("instructions", "")).lower()
        return "linux" in name or "linux" in role or "rhcsa" in instructions

    @staticmethod
    def _meets_threshold(confidence: str, threshold: str) -> bool:
        rank = {"none": 0, "low": 1, "medium": 2, "high": 3}
        return rank.get(confidence, 0) >= rank.get(threshold, 2)

    def _ensure_report(self) -> None:
        if self.report_path.exists():
            return
        self._write_report(
            {
                "created_at": dt.datetime.now().isoformat(),
                "api_calls_avoided": 0,
                "local_retrieval_hits": 0,
                "local_retrieval_misses": 0,
                "command_reuse_frequency": 0,
                "workflow_reuse": 0,
            }
        )

    def _read_report(self) -> dict[str, Any]:
        try:
            return json.loads(self.report_path.read_text(encoding="utf-8"))
        except (OSError, json.JSONDecodeError):
            return {}

    def _write_report(self, payload: dict[str, Any]) -> None:
        payload["updated_at"] = dt.datetime.now().isoformat()
        self.report_path.write_text(json.dumps(payload, indent=2, ensure_ascii=False), encoding="utf-8")
```

runtime/providers/__init__.py

- size: 178 bytes
- sha256: 54f0543270e1d8ea0b5cbb303825beefcd97fd33ea1e872665590a81f21813c0
- category: configuration

```
from .base import ModelProvider
from .aureon_provider import AureonProvider
from .config import ProviderManager
```

```
__all__ = ["ModelProvider", "AureonProvider", "ProviderManager"]
```

runtime/providers/aureon_provider.py

- size: 1212 bytes
- sha256: 003e9098f46ae135f6843a4a9f58e20e269452b3f92619ff176163bb3e259320
- category: configuration

```
from __future__ import annotations

import os

from .base import ModelProvider
from .openai_compatible import OpenAICompatibleProvider


class AureonProvider(ModelProvider):
    """Aureon-compatible cloud provider.

    This class intentionally has no offline fake responder. If no live backend
    is configured, the caller must fall back to another real cloud provider.
    """

    def __init__(self, model: str) -> None:
        super().__init__(provider="aureon", model=model)
        self._backend = self._build_backend(model)
        if self._backend is None:
            raise RuntimeError(
                "Aureon backend is not configured. Set AUREON_API_BASE_URL or use another provider."
            )

    def generate(self, prompt: str) -> str:
        return self._backend.generate(prompt)

    @staticmethod
    def _build_backend(model: str):
        base_url = os.getenv("AUREON_API_BASE_URL", "").strip().rstrip("/")
        api_key = os.getenv("AUREON_API_KEY", "").strip()

        if not base_url:
            return None

        return OpenAICompatibleProvider(
            provider="aureon",
            api_key=api_key,
            model=model,
            base_url=base_url,
        )
```

runtime/providers/base.py

- size: 380 bytes
- sha256: ec6d371c5a02c5deec3ea55107dc03261544d882412725ccaa910cc6607e6fee
- category: configuration

```
from __future__ import annotations

from dataclasses import dataclass
```

```python
@dataclass(frozen=True)
class ModelProvider:
    """Small provider interface used by the runtime."""

    provider: str
    model: str

    @property
    def full_name(self) -> str:
        return f"{self.provider}/{self.model}"

    def generate(self, prompt: str) -> str:
        raise NotImplementedError
```

runtime/providers/config.py

- size: 11492 bytes
- sha256: `af168e1970fc033abfbfd0d1c2420c48400740242cf0da893dace851981a259b`
- category: configuration

```python
from __future__ import annotations

import json
import os
from dataclasses import asdict, dataclass
from pathlib import Path

from .aureon_provider import AureonProvider
from .base import ModelProvider
from .gemini_provider import GeminiProvider
from .openai_compatible import OpenAICompatibleProvider


DEFAULT_MODEL = "openrouter/google/gemma-3-27b-it"
DEFAULT_PROVIDER_CHAIN = [
    {"name": "openrouter", "model": "google/gemma-3-27b-it", "enabled": True},
    {"name": "gemini", "model": "gemini-2.5-flash", "enabled": True},
    {"name": "deepseek", "model": "deepseek-chat", "enabled": True},
]

DEFAULT_MODEL_PRESETS: dict[str, str] = {
    "gemma": "openrouter/google/gemma-3-27b-it",
    "openrouter": "openrouter/google/gemma-3-27b-it",
    "openrouter-gemma": "openrouter/google/gemma-3-27b-it",
    "gemini": "gemini/gemini-2.5-flash",
    "deepseek": "deepseek/deepseek-chat",
    "aureon": "aureon/aureon-queen",
}

API_FILE_CANDIDATES = [
    Path.home() / ".config" / "openrouter" / "api.env",
    Path.home() / ".config" / "gemini" / "api.env",
    Path.home() / ".config" / "deepseek" / "api.env",
]

REMOVED_PROVIDERS = {"openai", "huggingface", "gemma-hf"}
```

```python
@dataclass
class ProviderConfig:
    name: str
    model: str
    enabled: bool = True

    @property
    def full_name(self) -> str:
        return f"{self.name}/{self.model}"


def load_api_environment() -> None:
    """Load known user API env files without exposing secrets."""
    for env_path in API_FILE_CANDIDATES:
        if not env_path.exists():
            continue
        values: dict[str, str] = {}
        for raw in env_path.read_text(encoding="utf-8", errors="ignore").splitlines():
            line = raw.strip()
            if not line or line.startswith("#") or "=" not in line:
                continue
            if line.startswith("export "):
                line = line[len("export ") :].strip()
            name, value = line.split("=", 1)
            name = name.strip()
            value = value.strip().strip('"').strip("'")
            if name and value:
                values[name] = value
        for name, value in values.items():
            if not os.getenv(name):
                os.environ[name] = value


class ProviderManager:
    """Cloud-provider manager with fallback routing and no fake offline mode."""

    def __init__(self, project_dir: Path) -> None:
        load_api_environment()
        self.project_dir = project_dir
        self.config_path = project_dir / "state" / "model_config.json"
        self.providers_path = project_dir / "state" / "providers.json"
        self.config_path.parent.mkdir(parents=True, exist_ok=True)
        self.provider_chain = self._load_provider_chain()
        self.current_model = self.normalize_model_name(self._load_model_name())
        self.provider: ModelProvider | None = None
        self.last_used_model = ""

    def generate(self, prompt: str) -> str:
        return self.generate_with_fallback(prompt)

    def generate_with_fallback(self, prompt: str) -> str:
        errors: list[str] = []
        tried: set[str] = set()
        for full_model in self._fallback_candidates():
            if full_model in tried:
```

```
            continue
        tried.add(full_model)
        try:
            provider = self._build_provider(full_model)
            response = provider.generate(prompt)
            self.provider = provider
            self.current_model = full_model
            self.last_used_model = provider.full_name
            return response
        except Exception as error:
            errors.append(f"{full_model}: {error}")

    if not errors:
        raise RuntimeError("No enabled cloud providers are configured.")

    raise RuntimeError(
        "No configured cloud provider succeeded. Checked:\n- " + "\n- ".join(errors)
    )

def switch_model(self, model_name: str) -> str:
    normalized = self.normalize_model_name(model_name)
    self.current_model = normalized
    self.provider = None
    self.config_path.write_text(
        json.dumps({"model": normalized}, indent=2),
        encoding="utf-8",
    )
    return self.current_model

def describe(self) -> str:
    return self.last_used_model or self.current_model

def active_fallback_chain(self) -> list[str]:
    return [provider.full_name for provider in self.provider_chain if provider.enabled]

def available_models(self) -> list[str]:
    return [
        f"{alias:<15} -> {model}"
        for alias, model in DEFAULT_MODEL_PRESETS.items()
        if model.split("/", 1)[0] not in REMOVED_PROVIDERS
        and self._provider_is_available(model.split("/", 1)[0])
    ]

def provider_status(self) -> list[dict[str, str | bool]]:
    rows: list[dict[str, str | bool]] = []
    for provider in self.provider_chain:
        rows.append(
            {
                "name": provider.name,
                "model": provider.model,
                "enabled": provider.enabled,
                "available": self._provider_is_available(provider.name),
                "full_name": provider.full_name,
            }
        )
    return rows
```

```
def model_notice(self, model_name: str) -> str | None:
    normalized = self.normalize_model_name(model_name)
    provider = normalized.split("/", 1)[0]
    if provider == "gemini":
        return "Gemini uses GEMINI_API_KEY and the google-genai SDK."
    if provider == "openrouter":
        return "OpenRouter uses OPENROUTER_API_KEY. Current Gemma preset: google/gemma-3-27b-it."
    if provider == "deepseek":
        return "DeepSeek uses DEEPSEEK_API_KEY and an OpenAI-compatible endpoint."
    if provider == "aureon":
        return "Aureon requires a live AUREON_API_BASE_URL. No offline fake mode is used."
    if provider in REMOVED_PROVIDERS:
        return "This provider was removed from the terminal app because it is not configured."
    return None

def normalize_model_name(self, model_name: str) -> str:
    value = model_name.strip()
    if not value:
        raise ValueError("Model name cannot be empty.")

    lowered = value.lower()
    if lowered in DEFAULT_MODEL_PRESETS:
        return DEFAULT_MODEL_PRESETS[lowered]

    if ":" in value and "/" not in value:
        provider, model = value.split(":", 1)
        value = f"{provider.strip()}/{model.strip()}"

    if "/" not in value:
        return f"gemini/{value}"

    provider, model = value.split("/", 1)
    provider = provider.strip().lower()
    model = model.strip()
    if not provider or not model:
        raise ValueError(f"Invalid model name: {model_name}")
    if provider in REMOVED_PROVIDERS:
        raise ValueError(f"Provider removed from terminal app: {provider}")
    return f"{provider}/{model}"

def _fallback_candidates(self) -> list[str]:
    candidates = [self.current_model]
    candidates.extend(
        provider.full_name
        for provider in self.provider_chain
        if provider.enabled and self._provider_is_available(provider.name)
    )
    return candidates

def _load_provider_chain(self) -> list[ProviderConfig]:
    if not self.providers_path.exists():
        providers = [ProviderConfig(**payload) for payload in DEFAULT_PROVIDER_CHAIN]
        self.providers_path.write_text(
            json.dumps({"providers": [asdict(provider) for provider in providers]}, indent=2),
            encoding="utf-8",
        )
        return providers
```

```
    try:
        payload = json.loads(self.providers_path.read_text(encoding="utf-8"))
    except json.JSONDecodeError:
        payload = {"providers": DEFAULT_PROVIDER_CHAIN}

    providers: list[ProviderConfig] = []
    for item in payload.get("providers", []):
        if not isinstance(item, dict):
            continue
        name = str(item.get("name", "")).strip()
        model = str(item.get("model", "")).strip()
        if name in REMOVED_PROVIDERS:
            continue
        if name and model:
            providers.append(
                ProviderConfig(
                    name=name,
                    model=model,
                    enabled=bool(item.get("enabled", True)),
                )
            )

    if not providers:
        return [ProviderConfig(**item) for item in DEFAULT_PROVIDER_CHAIN]

    existing = {(provider.name, provider.model) for provider in providers}
    for item in DEFAULT_PROVIDER_CHAIN:
        key = (item["name"], item["model"])
        if key not in existing:
            providers.append(ProviderConfig(**item))
    return providers

def _load_model_name(self) -> str:
    if not self.config_path.exists():
        return DEFAULT_MODEL
    try:
        payload = json.loads(self.config_path.read_text(encoding="utf-8"))
    except json.JSONDecodeError:
        return DEFAULT_MODEL
    model = str(payload.get("model") or DEFAULT_MODEL)
    provider = model.split("/", 1)[0]
    if provider in REMOVED_PROVIDERS:
        return DEFAULT_MODEL
    return model

def _build_provider(self, model_name: str) -> ModelProvider:
    provider, model = model_name.split("/", 1)

    if provider == "aureon":
        return AureonProvider(model)
    if provider == "gemini":
        api_key = os.getenv("GEMINI_API_KEY") or os.getenv("GOOGLE_API_KEY")
        if not api_key:
            raise FileNotFoundError("GEMINI_API_KEY not found")
        return GeminiProvider(api_key, model)
    if provider == "openrouter":
```

```
            return OpenAICompatibleProvider(
                provider="openrouter",
                api_key=self._load_env_key("OPENROUTER_API_KEY"),
                model=model,
                base_url="https://openrouter.ai/api/v1",
            )
        if provider == "deepseek":
            return OpenAICompatibleProvider(
                provider="deepseek",
                api_key=self._load_env_key("DEEPSEEK_API_KEY"),
                model=model,
                base_url=os.getenv("DEEPSEEK_BASE_URL", "https://api.deepseek.com/v1"),
            )
        if provider in REMOVED_PROVIDERS:
            raise ValueError(f"Provider removed from terminal app: {provider}")

        raise ValueError(f"Unsupported provider: {provider}")

    @staticmethod
    def _load_env_key(env_name: str) -> str:
        value = os.getenv(env_name, "").strip()
        if value:
            return value
        raise FileNotFoundError(f"{env_name} not found")

    @staticmethod
    def _provider_is_available(provider: str) -> bool:
        if provider == "aureon":
            return bool(os.getenv("AUREON_API_BASE_URL", "").strip())
        if provider == "gemini":
            return bool(os.getenv("GEMINI_API_KEY", "").strip() or os.getenv("GOOGLE_API_KEY", "").strip())
        if provider == "openrouter":
            return bool(os.getenv("OPENROUTER_API_KEY", "").strip())
        if provider == "deepseek":
            return bool(os.getenv("DEEPSEEK_API_KEY", "").strip())
        if provider in REMOVED_PROVIDERS:
            return False
        return False
```

runtime/providers/gemini_provider.py

- size: 890 bytes
- sha256: cc5a45c6d02be12549cdaea25d023e0a92357b945b14c45d45cbd4ab1aae870d
- category: configuration

```
from __future__ import annotations

import os

from .base import ModelProvider


class GeminiProvider(ModelProvider):
    """Gemini implementation of the small provider interface."""

    def __init__(self, api_key: str, model: str) -> None:
        try:
            from google import genai
```

```
        except ImportError as error:
            raise ImportError(
                "google-genai is required only when the Gemini provider is selected."
            ) from error

        os.environ["GEMINI_API_KEY"] = api_key
        os.environ.pop("GOOGLE_API_KEY", None)
        super().__init__(provider="gemini", model=model)
        self.client = genai.Client(api_key=api_key)

    def generate(self, prompt: str) -> str:
        response = self.client.models.generate_content(
            model=self.model,
            contents=prompt,
        )
        return (response.text or "").strip()
```

runtime/providers/gemma_provider.py

- size: 3858 bytes
- sha256: 4930ad95010bd878afa38f71217c2bb6274b37695d6280436c05cbe179f73e2a
- category: configuration

```
from __future__ import annotations

import json
import os
import urllib.error
import urllib.request

from .base import ModelProvider
from .openai_compatible import OpenAICompatibleProvider


class GemmaProvider(ModelProvider):
    """Gemma worker provider with local Ollama first and HF fallback."""

    def __init__(self, model: str = "gemma3:4b") -> None:
        super().__init__(provider="gemma", model=model)
        self.ollama_url = os.getenv("OLLAMA_BASE_URL", "http://127.0.0.1:11434").rstrip("/")
        self.hf_token = os.getenv("HF_TOKEN", "").strip() or os.getenv("HUGGINGFACE_API_KEY", "").strip()
        self.hf_model = os.getenv("GEMMA_HF_MODEL", "google/gemma-2-2b-it")
        self.openai_base_url = os.getenv("GEMMA_OPENAI_BASE_URL", "").strip().rstrip("/")
        self.openai_api_key = os.getenv("GEMMA_OPENAI_API_KEY", "").strip()

    def generate(self, prompt: str) -> str:
        errors: list[str] = []

        try:
            return self._generate_ollama(prompt)
        except Exception as error:
            errors.append(f"ollama: {error}")

        if self.hf_token:
            try:
                return self._generate_huggingface(prompt)
            except Exception as error:
                errors.append(f"huggingface: {error}")
```

```
    if self.openai_base_url:
        try:
            provider = OpenAICompatibleProvider(
                provider="gemma",
                api_key=self.openai_api_key,
                model=self.model,
                base_url=self.openai_base_url,
            )
            return provider.generate(prompt)
        except Exception as error:
            errors.append(f"openai-compatible: {error}")

    raise RuntimeError(
        "Gemma worker provider is not configured or reachable. Checked:\n- "
        + "\n- ".join(errors)
    )

def _generate_ollama(self, prompt: str) -> str:
    body = {
        "model": self.model,
        "prompt": prompt,
        "stream": False,
    }
    request = urllib.request.Request(
        f"{self.ollama_url}/api/generate",
        data=json.dumps(body).encode("utf-8"),
        headers={"Content-Type": "application/json"},
        method="POST",
    )
    with urllib.request.urlopen(request, timeout=45) as response:
        payload = json.loads(response.read().decode("utf-8"))
    return str(payload.get("response", "")).strip()

def _generate_huggingface(self, prompt: str) -> str:
    body = {
        "inputs": prompt,
        "parameters": {
            "max_new_tokens": 700,
            "temperature": 0.1,
            "return_full_text": False,
        },
    }
    request = urllib.request.Request(
        f"https://api-inference.huggingface.co/models/{self.hf_model}",
        data=json.dumps(body).encode("utf-8"),
        headers={
            "Authorization": f"Bearer {self.hf_token}",
            "Content-Type": "application/json",
        },
        method="POST",
    )
    try:
        with urllib.request.urlopen(request, timeout=90) as response:
            payload = json.loads(response.read().decode("utf-8"))
    except urllib.error.HTTPError as error:
        detail = error.read().decode("utf-8", errors="replace")
```

```
            raise RuntimeError(f"HF inference HTTP {error.code}: {detail}") from error

        if isinstance(payload, list) and payload:
            first = payload[0]
            if isinstance(first, dict):
                return str(first.get("generated_text", "")).strip()
        if isinstance(payload, dict):
            return str(payload.get("generated_text", "") or payload.get("text", "")).strip()
        raise RuntimeError(f"Unexpected HF inference payload: {payload}")
```

runtime/providers/openai_compatible.py

- size: 1674 bytes
- sha256: 78ab47e49a5c0ba94cef03aa99fabd05e74d08a6832ff2cf08c9af23c10452a2
- category: configuration

```
from __future__ import annotations

import json
import os
import urllib.error
import urllib.request

from .base import ModelProvider


class OpenAICompatibleProvider(ModelProvider):
    """Minimal OpenAI-compatible chat completions provider.

    This keeps provider switching independent from the agent runtime without
    adding another package dependency.
    """

    def __init__(
        self,
        provider: str,
        api_key: str,
        model: str,
        base_url: str,
    ) -> None:
        super().__init__(provider=provider, model=model)
        self.api_key = api_key
        self.base_url = base_url.rstrip("/")

    def generate(self, prompt: str) -> str:
        body = {
            "model": self.model,
            "messages": [{"role": "user", "content": prompt}],
            "temperature": 0,
            "max_tokens": int(os.getenv("OPENAI_COMPATIBLE_MAX_TOKENS", "1200")),
        }
        request = urllib.request.Request(
            f"{self.base_url}/chat/completions",
            data=json.dumps(body).encode("utf-8"),
            headers={
                "Authorization": f"Bearer {self.api_key}",
                "Content-Type": "application/json",
            },
```

```
            method="POST",
        )
        try:
            with urllib.request.urlopen(request, timeout=90) as response:
                payload = json.loads(response.read().decode("utf-8"))
        except urllib.error.HTTPError as error:
            detail = error.read().decode("utf-8", errors="replace")
            raise RuntimeError(f"{self.provider} HTTP {error.code}: {detail}") from error

        return payload["choices"][0]["message"]["content"].strip()
```

runtime/requirements.txt

- size: 39 bytes
- sha256: 042a8130c9b870e85789a3d1ddca591656b79219b7de3d1e11516b0c7aaf6c98
- category: runtime

```
google-genai>=1.0.0
playwright>=1.59.0
```

runtime/router/__init__.py

- size: 91 bytes
- sha256: e2d480d01056f1fd344f187268e5d46269e350718b9e73e6ddc18e45809d1d5e
- category: runtime

```
from .local_router import LocalRoute, LocalRouter

__all__ = ["LocalRoute", "LocalRouter"]
```

runtime/router/local_router.py

- size: 2954 bytes
- sha256: 77e470f9a826d2c49d93db84aef72895bcbcd411eb5dcc4c628fd40df6b3cc46
- category: runtime

```
from __future__ import annotations

import re
from dataclasses import dataclass
from pathlib import Path
from typing import Any


@dataclass
class LocalRoute:
    actions: list[dict[str, Any]]
    final_message: str | None = None


class LocalRouter:
    """Conservative local router for obvious tasks that do not need an LLM."""

    def __init__(self, desktop_dir: Path) -> None:
        self.desktop_dir = desktop_dir

    def route(self, user_input: str) -> LocalRoute | None:
        text = user_input.strip()
        lowered = text.lower()
```

```
        if not text:
            return None

        if self._asks_for_date(lowered):
            return LocalRoute([{"action": "shell_execute", "command": "date -Iseconds", "reason": "Local dat

        if lowered in {"pwd", "gdzie jestem", "pokaz katalog", "pokaz katalog"}:
            return LocalRoute([{"action": "shell_execute", "command": "pwd", "reason": "Local pwd route."}])

        if lowered in {"ls", "lista plikow", "lista plikow", "pokaz pliki", "pokaz pliki"}:
            return LocalRoute([{"action": "shell_execute", "command": "ls -la", "reason": "Local list route

        if lowered in {"curl --version", "sprawdz curl", "sprawdz curl"}:
            return LocalRoute([{"action": "shell_execute", "command": "curl --version", "reason": "Local cu

        folder_name = self._extract_desktop_folder_name(text)
        if folder_name:
            target = self.desktop_dir / folder_name
            return LocalRoute(
                [{"action": "create_folder", "path": str(target), "reason": "Local desktop folder route."}],
                f"Folder ready at {target}",
            )

        return None

    @staticmethod
    def _asks_for_date(lowered: str) -> bool:
        return (
            "jaki dzis" in lowered
            or "jaki dzis" in lowered
            or "data" == lowered
            or lowered in {"date", "dzisiejsza data"}
        )

    @staticmethod
    def _extract_desktop_folder_name(text: str) -> str | None:
        lowered = text.lower()
        if not any(token in lowered for token in ("folder", "katalog")):
            return None
        if not any(token in lowered for token in ("pulpit", "pulpicie", "desktop")):
            return None
        if not any(token in lowered for token in ("stw", "utw", "zrob", "zrob", "create", "make")):
            return None

        patterns = [
            r"(?:folder|katalog)\s+([A-Za-z0-9_.-]{2,80})",
            r"([A-Za-z0-9_.-]{2,80})\s+(?:na|on)\s+(?:pulpicie|desktop)",
        ]
        ignored = {"na", "on", "pulpicie", "desktop", "folder", "katalog"}
        for pattern in patterns:
            match = re.search(pattern, text, flags=re.IGNORECASE)
            if not match:
                continue
            name = match.group(1).strip().strip(".,?!")
            if name.lower() not in ignored:
                return name
        return None
```

runtime/run.sh

- size: 397 bytes
- sha256: 22a67e39bfcb9bd0b00931407f9a381ee0a5f133aa5cf1aef6bca3e2ecaeb5c6
- category: runtime

```
#!/usr/bin/env bash
set -euo pipefail

PROJECT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
VENV_PYTHON="$PROJECT_DIR/.venv/bin/python"
MAIN_FILE="$PROJECT_DIR/main.py"
VENV_DIR="$PROJECT_DIR/.venv"

if [[ -x "$VENV_PYTHON" ]]; then
  exec "$VENV_PYTHON" "$MAIN_FILE" "$@"
fi

echo "Virtual environment not found. Using system python3 for the local runtime."
exec python3 "$MAIN_FILE" "$@"
```

runtime/run_web.sh

- size: 364 bytes
- sha256: 56e4e37b5b69c960e791c21b806aa8a217679f2159e3b190eba2083390e1193c
- category: runtime

```
#!/usr/bin/env bash
set -euo pipefail

PROJECT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
VENV_PYTHON="$PROJECT_DIR/.venv/bin/python"
WEB_FILE="$PROJECT_DIR/webapp.py"

if [[ -x "$VENV_PYTHON" ]]; then
  exec "$VENV_PYTHON" "$WEB_FILE" "$@"
fi

echo "Virtual environment not found. Using system python3 for the web runtime."
exec python3 "$WEB_FILE" "$@"
```

runtime/webapp.py

- size: 5272 bytes
- sha256: 23d11e680b1acf77c360861c0e291c66a35cb2c2e24896d38581e448fabe8598
- category: runtime

```
#!/usr/bin/env python3
from __future__ import annotations

import json
import os
import traceback
from http import HTTPStatus
from http.server import SimpleHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from threading import Lock
from urllib.parse import urlparse

from main import (
```

```
    DEBUG_RAW_RESPONSE,
    PROMPT_FILE,
    AgentRuntime,
    ProviderManager,
    load_prompt_template,
)


PROJECT_DIR = Path(__file__).resolve().parent
WEB_DIR = PROJECT_DIR / "web"
HOST = os.getenv("APP2_WEB_HOST", "127.0.0.1")
PORT = int(os.getenv("APP2_WEB_PORT", "4311"))


class WebRuntimeService:
    """Shared runtime adapter used by the local web UI."""

    def __init__(self) -> None:
        self.runtime = AgentRuntime(
            provider_manager=ProviderManager(PROJECT_DIR),
            prompt_template=load_prompt_template(PROMPT_FILE),
            project_dir=PROJECT_DIR,
            debug_raw=DEBUG_RAW_RESPONSE,
        )
        self.lock = Lock()

    def status_payload(self) -> dict:
        payload = self.runtime.snapshot_status()
        payload["available_models"] = self.runtime.provider_manager.available_models()
        return payload

    def switch_model(self, model_name: str) -> dict:
        with self.lock:
            selected = self.runtime.provider_manager.switch_model(model_name)
            return {
                "ok": True,
                "model": selected,
                "notice": self.runtime.provider_manager.model_notice(selected),
                "status": self.status_payload(),
            }

    def run_prompt(self, prompt: str) -> dict:
        with self.lock:
            result = self.runtime.run_text_request(prompt)
            return {
                "ok": True,
                "transcript": result["transcript"],
                "status": result["status"],
            }


SERVICE = WebRuntimeService()


class CodexStyleHandler(SimpleHTTPRequestHandler):
    """Serve the static UI and a small JSON API."""
```

```
def __init__(self, *args, **kwargs):
    super().__init__(*args, directory=str(WEB_DIR), **kwargs)

def do_GET(self) -> None:
    parsed = urlparse(self.path)
    if parsed.path == "/api/status":
        self._write_json(HTTPStatus.OK, SERVICE.status_payload())
        return
    if parsed.path == "/api/models":
        self._write_json(
            HTTPStatus.OK,
            {
                "current_model": SERVICE.runtime.provider_manager.describe(),
                "available_models": SERVICE.runtime.provider_manager.available_models(),
            },
        )
        return
    if parsed.path in {"/", "/index.html"}:
        self.path = "/index.html"
    return super().do_GET()

def do_POST(self) -> None:
    parsed = urlparse(self.path)
    payload = self._read_json_body()
    if payload is None:
        return

    try:
        if parsed.path == "/api/chat":
            prompt = str(payload.get("prompt", "")).strip()
            if not prompt:
                self._write_json(HTTPStatus.BAD_REQUEST, {"ok": False, "error": "prompt is required"})
                return
            self._write_json(HTTPStatus.OK, SERVICE.run_prompt(prompt))
            return

        if parsed.path == "/api/model":
            model_name = str(payload.get("model", "")).strip()
            if not model_name:
                self._write_json(HTTPStatus.BAD_REQUEST, {"ok": False, "error": "model is required"})
                return
            self._write_json(HTTPStatus.OK, SERVICE.switch_model(model_name))
            return

        self._write_json(HTTPStatus.NOT_FOUND, {"ok": False, "error": "Not found"})
    except Exception as error:  # pragma: no cover - local debugging path
        self._write_json(
            HTTPStatus.INTERNAL_SERVER_ERROR,
            {
                "ok": False,
                "error": str(error),
                "traceback": traceback.format_exc(),
            },
        )

def log_message(self, format: str, *args) -> None:
    return
```

```
    def _read_json_body(self) -> dict | None:
        length = int(self.headers.get("Content-Length", "0") or 0)
        raw_body = self.rfile.read(length) if length else b"{}"
        try:
            return json.loads(raw_body.decode("utf-8"))
        except json.JSONDecodeError:
            self._write_json(HTTPStatus.BAD_REQUEST, {"ok": False, "error": "Invalid JSON body"})
            return None

    def _write_json(self, status: HTTPStatus, payload: dict) -> None:
        body = json.dumps(payload, ensure_ascii=False).encode("utf-8")
        self.send_response(status)
        self.send_header("Content-Type", "application/json; charset=utf-8")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)


def main() -> None:
    server = ThreadingHTTPServer((HOST, PORT), CodexStyleHandler)
    print(f"App222 web UI running on http://{HOST}:{PORT}")
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        pass
    finally:
        server.server_close()


if __name__ == "__main__":
    main()
```